

Linking (liên kết) là quá trình gom các mảnh code và dữ liệu để tạo thành 1 file có thể load vào bộ nhớ để thực thi. Quá trình liên kết có thể diễn ra ở thời điểm **Biên dịch**, khi mà mã nguồn được dịch thành mã máy, có thể ở thời điểm **Nạp**, khi file chương trình được nạp lên bộ nhớ (được thực thi bởi **Loader**). Hoặc là khi **Chạy** (được thực thi bởi chương trình ứng dụng). Với các hệ điều hành hiện đại, **Linking** được tự động hóa bởi các **Trình liên kết (Linkers)**.

1. Compile Time (Tại thời điểm biên dịch - Tĩnh)

- **Bản chất:** Trình liên kết (Linker) lấy các file đối tượng (`.o` hoặc `.obj`) từ code của bạn và các thư viện tĩnh (`.a` hoặc `.lib`), rồi "dán" (merge) tất cả chúng lại thành một file thực thi duy nhất (`.exe` hoặc ELF binary).
- **Đặc điểm:** Sau khi xong bước này, chương trình đã chứa mọi thứ nó cần. Nó không còn phụ thuộc vào các file rời rạc bên ngoài để chạy nữa.
- **Trong RE:** Đây là lý do bạn thường thấy các hàm chuẩn (như `printf`) đã nằm sẵn bên trong file thực thi.

2. Load Time (Tại thời điểm nạp - Động)

- **Bản chất:** Khi bạn nhấn đúp vào file `.exe` để chạy, hệ điều hành sẽ khởi động `Loader` . Loader này sẽ tìm các thư viện chia sẻ (shared libraries như `.dll` trên Windows hoặc `.so` trên Linux) mà chương trình cần, rồi nạp chúng vào bộ nhớ và "nối" địa chỉ các hàm từ thư viện vào chương trình của bạn.
- **Đặc điểm:** File thực thi trên đĩa không chứa mã nguồn của thư viện đó, nó chỉ chứa một "danh sách yêu cầu" (Import Table).
- **Trong RE:** Đây là lúc bạn sẽ thấy bảng Import (Import Table) hoạt động. Loader sẽ quyết định thư viện nào được nạp vào địa chỉ nào trong bộ nhớ.

3. Run Time (Tại thời điểm thực thi - Động hoàn toàn)

- **Bản chất:** Chương trình đang chạy rồi, nhưng nó tự ý quyết định: "Tôi cần thêm thư viện này ngay bây giờ". Nó sẽ gọi các hàm hệ thống (như `LoadLibrary` trên Windows hoặc `dlopen` trên Linux) để tự nạp thư viện vào bộ nhớ và sử dụng.
- **Đặc điểm:** Rất linh hoạt. Chương trình có thể chạy mà không cần thư viện đó cho đến khi một tính năng cụ thể được kích hoạt.
- **Trong RE:** Đây là "kẻ thù" của phân tích tĩnh. Vì không thể thấy thư viện nào được nạp trước khi chạy, bạn buộc phải dùng Dynamic Analysis (debugger) để xem lúc nào chương trình gọi hàm `dlopen` hay `LoadLibrary` .

Linkers đóng vai trò quan trọng trong phát triển phần mềm vì nó cho phép **Seperate compilation**, nghĩa là thay vì tổ chức chương trình theo cấu trúc nguyên khối, ta có thể chia nhỏ ra thành các **Module**, chúng có thể được sửa đổi và biên dịch độc lập với nhau. Khi chỉnh sửa 1 **Module**, ta chỉ cần liên kết và biên dịch lại nó, thay vì phải liên kết và biên dịch lại toàn bộ chương trình.

<pre>(a) main.c ----- code/link/main.c 1 int sum(int *a, int n); 2 3 int array[2] = {1, 2}; 4 5 int main() 6 { 7 int val = sum(array, 2); 8 return val; 9 } ----- code/link/main.c</pre>	<pre>(b) sum.c ----- code/link/sum.c 1 int sum(int *a, int n) 2 { 3 int i, s = 0; 4 5 for (i = 0; i < n; i++) { 6 s += a[i]; 7 } 8 return s; 9 } ----- code/link/sum.c</pre>
---	---

Figure 7.1 Example program 1. The example program consists of two source files, `main.c` and `sum.c`. The `main` function initializes an array of ints, and then calls the `sum` function to sum the array elements.

1. Compiler Driver (Trình điều khiển biên dịch).

Hầu hết các hệ thống biên dịch đều cung cấp **Compiler driver**, nó gọi đến các bộ **tiền xử lý ngôn ngữ (Language Preprocessor)**, **trình biên dịch (Compiler)**, và **trình liên kết (Linker)**, theo nhu cầu của người dùng. Ví dụ, muốn build 1 chương trình C bằng hệ thống biên dịch **GNU**, chúng ta phải gọi đến trình điều khiển biên dịch **GCC** bằng lệnh `gcc file.c -o`

1. Tại sao mỗi ngôn ngữ lại có một hệ thống biên dịch riêng?

Mỗi ngôn ngữ lập trình có một cú pháp (syntax) và quy tắc (grammar) riêng. Trình biên dịch (Compiler) được thiết kế để "hiểu" ngôn ngữ đó và chuyển nó thành mã máy cho một kiến trúc CPU cụ thể (ví dụ: x86-64, ARM).

- C++ cần `g++` hoặc `clang++` hoặc `MSVC` .
- Rust cần `rustc` .
- Go cần trình biên dịch `go` .
- Java cần `javac` (nó biên dịch ra bytecode thay vì mã máy trực tiếp).

Vì logic của các ngôn ngữ này khác nhau hoàn toàn, nên không thể dùng trình biên dịch C++ để dịch code Rust được.

2. "Tải về" thực chất là gì?

Khi bạn "tải về" một hệ thống biên dịch, thực chất bạn đang cài đặt một Bộ công cụ phát triển (Toolchain).

- Trên Windows: Bạn thường cài một IDE như *Visual Studio*. Nó đã đóng gói sẵn "tất cả trong một": trình biên dịch, trình liên kết, trình sửa lỗi (debugger).
- Trên Linux: Bạn thường dùng trình quản lý gói để cài. Ví dụ, để có hệ thống biên dịch cho C/C++, bạn chạy:

```
sudo apt install build-essential
```

Lệnh này sẽ tải về cả một "hệ thống" gồm `gcc` (trình biên dịch), `g++` , `make` (công cụ quản lý biên dịch), `ld` (linker), và nhiều công cụ hỗ trợ khác.

3. Bạn có phải tự cài từng thành phần không?

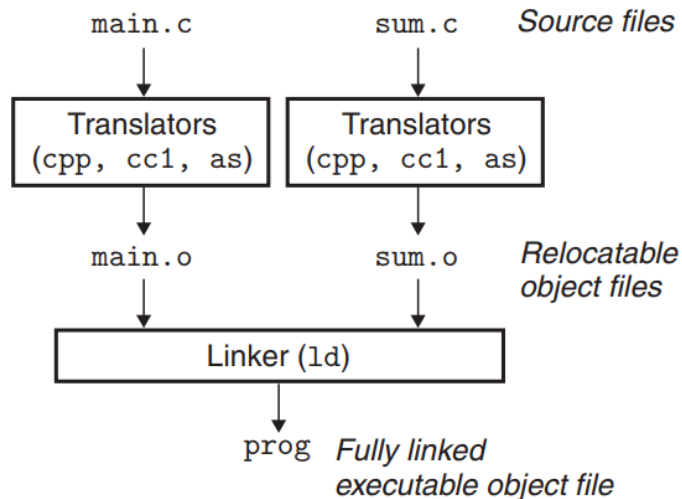
Không. Hầu hết các ngôn ngữ hiện đại đều cung cấp một "Trình điều khiển" (Driver/Manager) rất mạnh mẽ:

- Python: Bạn cài "Python interpreter", nó đóng vai trò vừa là trình dịch, vừa là môi trường chạy.
- Rust: Bạn dùng `cargo` . Bạn chỉ cần cài `cargo` , nó sẽ tự quản lý việc tải trình biên dịch, tải thư viện, và tự động gọi trình biên dịch cho bạn.
- Node.js (JavaScript): Bạn cài `node` , nó đi kèm với engine V8 (chính là trình biên dịch JIT cho JS).

Hình 7.2 mô tả quá trình biên dịch file code thành file đối tượng thực thi (trong ví dụ của hình 7.1).

Figure 7.2

Static linking. The linker combines relocatable object files to form an executable object file prog.



- Đầu tiên, driver gọi đến **C preprocessor (cpp)** để biên dịch file `main.c` thành file trung gian `main.i`
`cpp [other arguments] main.c /tmp/main.i`
- Tiếp theo, driver gọi đến **C compiler (cc1)** để biên dịch file `main.i` thành file hợp ngữ `main.s`
`cc1 /tmp/main.i -Og [other arguments] -o /tmp/main.s`
- Sau đó, driver gọi đến **Assembler (as)** để biên dịch file `main.s` thành file **đối tượng nhị phân tái định vị (binary relocate object file)** `main.o`.
`as [other arguments] -o /tmp/main.o /tmp/main.s`
- Driver cũng thực hiện tương tự với file `sum.c`. Sau khi có file `main.o` và `sum.o`, **Linker** sẽ ghép nối 2 file, cùng với các file hệ thống cần thiết khác, để tạo ra file **đối tượng thực thi (executable object file)**, tên là `prog`.
`ld -o prog [system object files and args] /tmp/main.o /tmp/sum.o`

- Để chạy file, gõ `./prog` trên Terminal.

shell với terminal khác gì nhau

Để hiểu đơn giản: Terminal là cái "vỏ" (phần cứng/cửa sổ), còn Shell là "bộ não" (phần mềm/bộ xử lý) ở bên trong.

Hãy tưởng tượng bạn đang ở trong một nhà hàng:

- Terminal giống như cái bàn ăn hoặc thực đơn mà bạn tương tác. Đó là nơi bạn gõ lệnh vào.
- Shell giống như người phục vụ. Anh ta nghe những gì bạn yêu cầu (lệnh), mang yêu cầu đó vào nhà bếp (Hệ điều hành), rồi lấy kết quả đem ra lại cho bạn.

1. Terminal (Trình mô phỏng thiết bị đầu cuối)

Đây là chương trình cung cấp giao diện để bạn nhập liệu.

- **Nhiệm vụ:** Chỉ đơn thuần là một "cửa sổ" để bạn gõ phím và hiển thị kết quả trả về. Nó không tự mình thực thi các lệnh.
- **Ví dụ:** Windows Terminal, PuTTY, iTerm2, hay cái cửa sổ đen đen mà bạn đang mở trong WSL.
- **Nếu không có Terminal:** Bạn vẫn có thể giao tiếp với máy tính, nhưng phải dùng những cách phức tạp hơn (ví dụ: qua cáp nối tiếp hoặc remote trực tiếp vào nhân hệ điều hành).

2. Shell (Bộ thông dịch dòng lệnh)

Đây là chương trình thực sự hiểu những gì bạn gõ.

- **Nhiệm vụ:** Nhận lệnh từ Terminal, phân tích cú pháp của lệnh đó, và gửi yêu cầu tới Kernel (Nhân hệ điều hành) để thực thi.
- **Ví dụ:** `bash`, `zsh`, `sh`, `powershell`, `cmd`.
- **Nếu không có Shell:** Bạn sẽ không thể ra lệnh bằng văn bản cho máy tính được. Bạn sẽ phải tương tác với hệ điều hành thông qua các chương trình khác (như GUI - giao diện đồ họa).

- Linux shell sẽ gọi đến 1 chương trình hệ thống gọi là **Loader**. Nhiệm vụ của nó là sao chép code và dữ liệu trong file thực thi vào bộ nhớ, và chuyển quyền điều khiển vào vị trí bắt đầu của chương trình.

2. Static Linking (liên kết tĩnh).

Các trình liên kết tĩnh (Static Linkers) như **Linux LD** nhận đầu vào là các file đối tượng tái định vị và các tham số dòng lệnh để sinh ra đầu ra là 1 chương trình đối tượng thực thi được liên kết đầy đủ, có thể load và chạy được.

Các file đối tượng đầu vào được tạo thành từ nhiều các **section** dữ liệu và code, mỗi

section là một chuỗi các byte liên tục. Các lệnh ở trong 1 **section**, các biến khởi tạo toàn cục nằm trong 1 **section** khác, và các biến chưa được khởi tạo lại nằm trong 1 **section** khác nữa...

Để build được 1 file thực thi, **Linker** cần làm 2 việc chính:

- **Phân giải kí hiệu (Symbol Resolution):** Các file đối tượng định nghĩa và tham chiếu các **Symbol**, mỗi **Symbol** là tên của 1 hàm, 1 biến... **Định nghĩa symbol** chính là chỉ đến nơi mà **symbol** đó được cấp phát bộ nhớ. Còn **tham chiếu symbol** là những nơi mà nó được gọi. Nhiệm vụ của việc phân giải là tìm địa chỉ định nghĩa **symbol** đó, để gán vào những nơi tham chiếu **symbol**.

2. "Phân giải" nghĩa là gì?

Linker sẽ đi rà soát toàn bộ các file `.o` để thực hiện một việc: Tìm "người yêu" cho các tham chiếu.

- **Vấn đề:** File `A.o` có một dòng lệnh: "Tôi cần gọi hàm `calculate()`".
- **Câu hỏi:** Hàm `calculate()` đó nằm ở đâu?
- **Việc của Linker:** Linker sẽ lục lọi trong danh sách các định nghĩa của tất cả các file (`A.o`, `B.o`, `C.o` và cả các thư viện chuẩn).
- **Kết quả:** Nếu nó tìm thấy `calculate()` nằm trong `B.o`, nó sẽ "phân giải" (resolve) bằng cách dán địa chỉ của hàm trong `B.o` vào chỗ mà `A.o` đang cần.

- **Tái định vị (Relocation):** Trình biên dịch (Compiler) và trình hợp dịch (Assembler) có nhiệm vụ sinh ra các section code và dữ liệu bắt đầu từ địa chỉ 0. **Linker** tái định vị chúng bằng cách gán mỗi **địa chỉ section** với 1 **định nghĩa symbol**. Sau đó chỉnh sửa tất cả các **tham chiếu symbol** thành địa chỉ trỏ đến vùng nhớ của **symbol** đó. **Linker** thực hiện việc này thông qua các **lệnh thông tin** do **Assembler** cung cấp. Chúng được gọi là **Chỉ mục (relocation entry)**, chứa thông tin về địa chỉ, cùng 1 số thông tin khác phục vụ cho việc tái định vị.

3. Object file (file đối tượng).

Object file có 3 loại:

- **Relocatable object file:** Chứa mã nhị phân và dữ liệu. Các file này có thể kết hợp với nhau tại thời điểm biên dịch để tạo ra file **Executable object file**.
- **Executable object file:** Chứa mã nhị phân và dữ liệu ở dạng có thể copy trực tiếp vào bộ nhớ để thực thi.
- **Shared object file:** Một dạng đặc biệt của **Relocatable object file**. Nó có thể được load vào bộ nhớ và liên kết động (nghĩa là chương trình sẽ gọi nó khi cần thay vì ghép trực tiếp vào file thực thi), ở cả load time và run time.
Compilers và assembler tạo ra **Relocatable object file** (kể cả **Shared object file**).

Linkers tạo ra **Executable object file**. Về mặt kỹ thuật, một **Object module** là một chuỗi các byte liên tục, và **Object file** là 1 **Object module** được lưu trên đĩa thành 1 file. 1 **Object file** được tổ chức dựa trên **Object file format** (mỗi hệ điều hành có 1 format khác nhau). Hệ điều hành Unix sử dụng format **a.out**, Windows dùng PE, Linux dùng ELF... Mặc dù tài liệu này chỉ tập chung về file ELF, các khái niệm này đối với các format khác cũng tương tự nhau.

4. Relocatable Object file (File đối tượng tái định vị).

Để hiểu chữ "tái định vị" (relocatable), bạn phải đặt mình vào vị trí của Trình hợp dịch (Assembler) khi nó dịch một file code đơn lẻ (ví dụ: `main.c`) thành file đối tượng (`main.o`).

Khi dịch riêng lẻ như vậy, Assembler hoàn toàn mù tịt về tổng thể chương trình. Nó không hề biết:

1. File thực thi cuối cùng sẽ lớn bao nhiêu byte.
2. Hàm `main` sẽ được đặt ở địa chỉ nào trong RAM khi chạy.
3. Các hàm ở file khác (như hàm `printf` của hệ thống) nằm ở đâu.

Cách giải quyết của Assembler: "Đặt địa chỉ giả"

Vì không biết địa chỉ thật, Assembler chơi bài **giả định**: Nó coi như file `main.o` này là duy nhất và cho mọi địa chỉ bên trong nó bắt đầu từ số 0 (địa chỉ gốc tạm thời).

- o Nếu trong code bạn gọi một hàm `đọc_file()`, Assembler không biết hàm này nằm ở đâu, nó sẽ tạm thời điền địa chỉ của lệnh gọi đó là `0x00000000`.

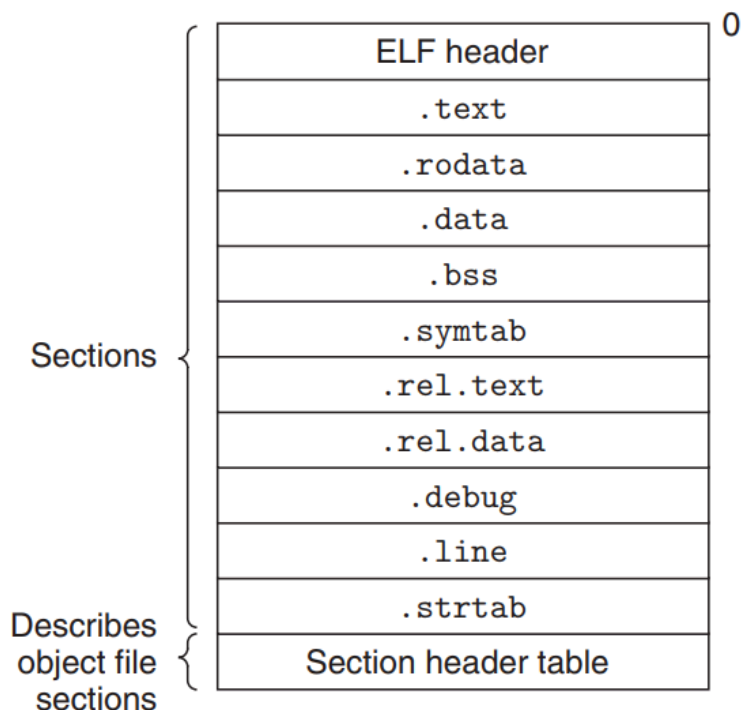
Vì tất cả địa chỉ bên trong file `.o` đều là địa chỉ giả/địa chỉ tạm thời, nên file này được gọi là **Có thể tái định vị**. Nghĩa là các địa chỉ trong này có thể dịch chuyển, thay đổi vị trí được.

Quá trình "Tái định vị" (Relocation) thực tế của Linker

Khi bạn đưa tất cả các file `.o` cho Linker, Linker sẽ làm nhiệm vụ gom chúng lại và thực hiện việc "tái định vị" theo các bước:

1. **Gom các section:** Linker nối đuôi các section lại với nhau. Giả sử khối `.text` của `file1.o` dài 1000 bytes và được đặt ở địa chỉ gốc `0x1000`. Khối `.text` của `file2.o` sẽ được xếp ngay sau đó, tức là bắt đầu từ địa chỉ `0x2000`.
2. **Tính toán địa chỉ thật:** Lúc này, mọi hàm và biến đã có một vị trí thực sự chính xác trong file thực thi cuối cùng.
3. **Sửa địa chỉ giả thành địa chỉ thật:** Linker sẽ dò theo một danh sách "ghi chú" mà Assembler để lại (gọi là **Relocation Table**) và sửa lại toàn bộ các địa chỉ `0x00000000` (giả) thành các địa chỉ thật (ví dụ: `0x1000` hoặc `0x2000`).

Figure 7.3
Typical ELF relocatable
object file.



Trong hình là cấu trúc của 1 ELF Relocatable object file cơ bản (chi tiết nằm trong file khác, ở đây sẽ không dịch thêm).

5. Symbol và Symbol Table.

Mỗi Relocatable object module, `m`, chứa 1 Symbol table bao gồm các thông tin về các module được định nghĩa và tham chiếu bởi `m`. Trong ngữ cảnh của 1 Linker, có 3 loại symbols:

- **Global symbol:** Được định nghĩa bởi module `m` và có thể được tham chiếu bởi các module khác. Global symbol tương ứng với các hàm được khai báo `nonstatic` và các biến toàn cục.
- **External symbol:** Là các Global symbol được tham chiếu bởi module `m` nhưng lại được định nghĩa bởi một module khác. External symbol tương ứng với các hàm `nonstatic` và các biến toàn cục nhưng được khai báo ở module khác.
- **Local symbol:** Được định nghĩa và tham chiếu duy nhất bởi module `m`. Nó tương ứng với các hàm `static` trong C và các biến toàn cục được khai báo bằng `static`. Các symbol này có thể được nhìn thấy từ bất kì đâu trong nội bộ module `m`, nhưng không module nào khác có thể tham chiếu nó.

Cần phân biệt rõ ràng rằng Local symbol khác với Local variable. Section `.symtab` (Symbol table) không chứa bất kì một biến cục bộ, nonstatic nào. Chúng được quản lí bằng stack khi chạy và không liên quan gì đến linker.

Ngoài ra, các biến cục bộ được khai báo bằng `static` lại không được quản lí bởi stack. Trình biên dịch sẽ cấp phát bộ nhớ cho chúng ở trong section `.bss` và `.data` với mỗi 1 định nghĩa, và tạo ra các Local symbol trong `.symtab` với các tên đặc biệt.

```
1  int f()  
2  {  
3      static int x = 0;  
4      return x;  
5  }  
6  
7  int g()  
8  {  
9      static int x = 1;  
10     return x;  
11 }
```

Như ví dụ trong hình, 2 biến `x` được khai báo cục bộ trong 2 hàm khác nhau với thuộc tính `static`. Trong trường hợp này, Compiler sẽ tạo ra 2 **Local symbol** với tên khác nhau. VD, nó sẽ dùng `x.1` cho biến `x` trong hàm `f()` và `x.2` cho biến `x` trong hàm `g()`.

1. Đối với Biến (Variables)

Biến Non-static (Biến thông thường / Biến cục bộ)

- **Trong code:** Là các biến bạn khai báo bình thường bên trong một hàm (ví dụ: `int x = 10;`).
- **Tuổi thọ & Bộ nhớ:** Nó chỉ sống tạm thời trên vùng nhớ Stack. Khi hàm được gọi, biến được sinh ra; khi hàm chạy xong và thoát (pop stack frame), biến đó bị xóa sổ hoàn toàn khỏi RAM.
- **Góc nhìn RE:** Bạn sẽ thấy nó được truy xuất qua thanh ghi `rbp` hoặc `rsp` (ví dụ: `mov [rbp-4], 10`).

Biến Static (Biến tĩnh)

- **Trong code:** Được khai báo kèm từ khóa `static` . Có hai loại:
 - **Static cục bộ (trong hàm):** Giúp biến giữ lại giá trị cũ giữa các lần gọi hàm khác nhau. Nó chỉ được khởi tạo đúng một lần duy nhất khi chương trình bắt đầu chạy.
 - **Static toàn cục (ngoài hàm):** Giới hạn phạm vi của biến đó chỉ được phép sử dụng trong nội bộ file `.c` đó. Các file `.c` khác không thể nhìn thấy hay can thiệp vào.
- **Tuổi thọ & Bộ nhớ:** Nó sống vĩnh viễn suốt toàn bộ vòng đời của chương trình. Nó được Linker xếp riêng vào các section dữ liệu: `.data` (nếu có khởi tạo giá trị) hoặc `.bss` (nếu không khởi tạo).
- **Góc nhìn RE:** Nó có một địa chỉ nhớ cố định tuyệt đối trong phân đoạn dữ liệu, chứ không nằm trên Stack.

2. Đối với Hàm (Functions)

Trong C, mặc định tất cả các hàm đều là **Non-static**.

Hàm Non-static (Hàm mặc định)

- **Bản chất:** Là hàm có tính "toàn cầu" (Global).
- **Góc nhìn Linker:** Linker sẽ đưa tên hàm này vào bảng *Symbol Table* dưới dạng *Global Symbol*. Nhờ vậy, file `file2.c` chỉ cần dùng từ khóa `extern` là có thể gọi được hàm này từ `file1.c` .

Hàm Static (Hàm tĩnh)

- **Bản chất:** Là hàm bị "bế quan tỏa cảng", chỉ được phép gọi bởi các hàm khác nằm trong cùng một file `.c` với nó.
- **Góc nhìn Linker:** Linker sẽ đánh dấu hàm này là *Local Symbol*. Khi Linker thực hiện bước *Symbol Resolution* (Phân giải ký hiệu) mà chúng ta vừa học ở trên, nó sẽ nghiêm cấm không cho các file `.c` bên ngoài tham chiếu tới hàm này. Nếu file khác cố tình gọi, Linker sẽ báo lỗi "undefined reference" ngay lập tức.

```

1  #include <stdio.h>
2
3  static int g_static = 100; // STATIC TOÀN CỤC: Chỉ file này dùng được. Sống
    vĩnh viễn (.data).
4  int g_normal = 200;        // NON-STATIC TOÀN CỤC: File khác dùng ké được.
    Sống vĩnh viễn (.data).
5
6  void test() {
7      static int l_static = 0; // STATIC CỤC BỘ: Giữ nguyên giá trị sau khi
    thoát hàm. Sống vĩnh viễn (.data).
8      int l_normal = 0;        // NON-STATIC CỤC BỘ: Reset về 0 mỗi khi vào
    hàm. Sống tạm thời (Stack).
9
10     l_static++;
11     l_normal++;
12     printf("Static: %d | Normal: %d\n", l_static, l_normal);
13 }
14
15 int main() {
16     test(); // In ra: Static: 1 | Normal: 1
17     test(); // In ra: Static: 2 | Normal: 1
18     test(); // In ra: Static: 3 | Normal: 1
19     return 0;
20 }

```

Symbol table được tạo ra bởi Assembler, sử dụng các symbol được Compiler xuất ra trong file hợp ngữ `.s`. 1 **ELF symbol table** được lưu trong section `.symtab`. Nó chứa 1 mảng các chỉ mục. Hình 7.4 cho ta thấy định dạng của mỗi chỉ mục.

code/link/elfstructs.c

```

1  typedef struct {
2      int     name;        /* String table offset */
3      char    type:4,      /* Function or data (4 bits) */
4              binding:4;   /* Local or global (4 bits) */
5      char    reserved;    /* Unused */
6      short   section;     /* Section header index */
7      long    value;       /* Section offset or absolute address */
8      long    size;        /* Object size in bytes */
9  } Elf64_Symbol;

```

code/link/elfstructs.c

Figure 7.4 ELF symbol table entry. The type and binding fields are 4 bits each.

Mỗi chỉ mục được tổ chức thành 1 cấu trúc với các trường thông tin.

- **name** : chứa địa chỉ offset trỏ đến tên symbol trong **String table (.strtab)**.
- **value** là địa chỉ trỏ đến symbol. Với **Relocatable object file**, nó là địa chỉ offset so với đầu section chỉ đến nơi symbol được định nghĩa. Còn với **Executable object file**, nó chính là địa chỉ tuyệt đối trỏ đến nơi symbol được định nghĩa trong bộ nhớ.

Absolute run-time address (Địa chỉ tuyệt đối lúc chạy) là địa chỉ bộ nhớ RAM thực tế và chính xác mà biến hoặc hàm đó sẽ chiếm giữ khi chương trình được nạp lên và bắt đầu chạy.

Để hiểu cụ thể, hãy so sánh sự khác biệt của trường **value** (giá trị địa chỉ) trong cấu trúc Symbol qua 2 giai đoạn mà đoạn sách đề cập:

1. Trong file .o (Relocatable module)

- **Value = Offset:** Vì chưa biết chương trình sẽ nằm ở đâu trong RAM, **value** chỉ là khoảng cách tính từ đầu section.
- **Ví dụ:** Hàm `main` có `value = 0x50`. Nghĩa là nó nằm cách điểm bắt đầu của section `.text` đúng 80 bytes. Đây chưa phải là địa chỉ để CPU có thể nhảy tới chạy được.

2. Trong file chạy được (Executable object file)

Sau khi Linker gom hết code và ấn định vị trí cố định cho chương trình, các địa chỉ tạm thời (offset) sẽ được biến đổi thành **Run-time address**.

- **Value = Absolute run-time address:** Linker điền thẳng địa chỉ RAM thực tế vào đây.
- **Ví dụ:** Giả sử section `.text` được nạp vào RAM tại địa chỉ `0x400000`. Lúc này, địa chỉ của hàm `main` sẽ được Linker tính toán và ghi lại là:

$$0x400000 + 0x50 = 0x400050$$

- Khi bạn double-click chạy file, Hệ điều hành nạp đúng hàm `main` vào ô nhớ `0x400050` trên RAM. CPU chỉ việc trở thành ghi lệnh (`rip`) thẳng vào địa chỉ tuyệt đối này để thực thi.

- **size** là kích thước của object (là đối tượng được định nghĩa là symbol) đó.

Bạn viết câu này đúng hoàn toàn về mặt bản chất. Tác giả dùng từ "object" ở đây với nghĩa chung chung là "đối tượng bộ nhớ", và việc bạn mở ngoặc giải thích "*là đối tượng được định nghĩa là symbol đó*" giúp làm rõ nghĩa rất tốt.

Để hiểu sâu hơn vì sao trường này tồn tại và ứng dụng của nó trong RE, hãy nhìn vào kích thước thực tế của từng loại symbol:

- **Nếu symbol là một biến (Data):** `size` chính là số byte mà kiểu dữ liệu của biến đó chiếm dụng trong RAM.
 - Ví dụ: `int x;` → `size` là 4 byte. `double y;` → `size` là 8 byte. Nếu là một mảng `int arr[10];` → `size` sẽ là 40 byte.
- **Nếu symbol là một hàm (Function):** `size` chính là tổng độ dài (số byte) của toàn bộ các lệnh mã máy cấu thành nên hàm đó trong section `.text`.
 - Ví dụ: Hàm `main` bắt đầu từ địa chỉ `0x400050` và kết thúc ở `0x400080` → `size` của nó sẽ là `0x30` (48 byte).

Tại sao Linker và các công cụ RE (như IDA, Ghidra) cực kỳ cần trường `size` này?

1. **Linker kiểm tra tràn bộ nhớ:** Khi gom các file lại, Linker dựa vào `size` để biết thực tế symbol này cần bao nhiêu đất trong phân đoạn dữ liệu, tránh việc các biến đè lên nhau.
2. **IDA/Ghidra dựng lại cấu trúc hàm:** Khi bạn ném một file binary vào IDA Pro, làm sao IDA biết một hàm bắt đầu từ đâu và kết thúc ở đâu để vẽ ra sơ đồ khối (Graph view) cho bạn xem? Chính là nhờ nó đọc trường `value` (địa chỉ bắt đầu) và cộng thêm trường `size` (độ dài) từ bảng Symbol Table này ra đấy.

- **type** quy định symbol này định nghĩa 1 hàm, dữ liệu, section...
- **binding** cho biết symbol là local hay global.
- **section** trỏ vào bảng **Section header table**, cho biết symbol đó đang nằm trong section nào.

Symbol table có thể chứa cả chỉ mục đến các section hoặc đường dẫn đến file nguồn gốc.

Ý này trong sách muốn nói rằng: Bảng Symbol Table (`.symtab`) không chỉ lưu mỗi tên hàm hay tên biến của bạn, mà nó còn chứa các bản ghi đặc biệt phục vụ cho chính thằng Linker và Debugger.

Cụ thể, hai loại bản ghi bổ sung đó là:

1. Chỉ mục đến các section (Entries for individual sections)

Như mình có nói ở phần `type ( STT_SECTION )`, bản thân mỗi section (như `.text`, `.data`, `.rodata`) cũng sẽ có một dòng đại diện riêng nằm trong bảng Symbol Table.

- **Mục đích:** Khi Linker thực hiện quá trình tái định vị (Relocation), nó cần biết vị trí gốc của cả một phân đoạn để làm mốc tính toán toán học, dịch chuyển toàn bộ các hàm/biến nằm bên trong phân đoạn đó đi theo.

2. Đường dẫn đến file nguồn gốc (Entries for original source file name)

Đây là các bản ghi có `type` là `STT_FILE`. Trường `name` của nó sẽ trỏ đến một chuỗi văn bản chính là tên file code ban đầu của bạn (ví dụ: `"main.c"`, `"utils.c"`).

- **Mục đích:** Phục vụ cho các công cụ gỡ lỗi như GDB. Khi chương trình bị crash hoặc khi bạn đặt breakpoint tại một hàm, GDB sẽ nhìn vào bảng Symbol Table để biết chính xác hàm này được biên dịch ra từ file nguồn nào trên ổ cứng, từ đó lỗi file `.c` đó ra hiển thị dòng code lỗi cho bạn thấy.

Tóm lại ngắn gọn: Symbol table là một cái "kho quản lý danh tính". Ngoài việc quản lý danh tính của các đối tượng do bạn tạo ra (hàm, biến), nó phải quản lý luôn danh tính của các thành phần hệ thống (phân đoạn) và nguồn gốc xuất xứ (file `.c`) để Linker và Debugger làm việc được suôn sẻ.

Mỗi **symbol** được gán (định nghĩa, tham chiếu) trong nhiều **section** của 1 file đối tượng, và nó được biểu thị bằng trường `section`, nó chứa chỉ số của **section** đó trong **section header table**. Có 3 loại **pseudosections (giả section)** không có chỉ mục trong section header table:

- **ABS:** cho các symbol không được cho phép tái định vị.
- **UNDEF:** cho các symbol được tham chiếu trong file đối tượng, nhưng được định nghĩa trong file khác.
- **COMMON:** cho các đối tượng dữ liệu chưa được khởi tạo, nhưng được định nghĩa ở đâu đó trong file đối tượng.

Cần chú ý rằng, **pseudosections** chỉ có trong file đối tượng tái định vị, chứ không có trong file đối tượng thực thi.

Giữa pseudo section **COMMON** và section `.bss` có nhiều điểm tương đồng nhất định. Các phiên bản hiện đại của GCC gán symbol trong các file tái định vị cho **COMMON** và `.bss` bằng cách sử dụng quy tắc sau:

- **COMMON:** cho các biến toàn cục chưa khởi tạo.

- `.bss` : cho các biến static chưa khởi tạo, và các biến toàn cục hoặc static được khởi tạo bằng 0.
- `READELF` là một công cụ hữu ích trên linux để xem nội dung của file đối tượng. Ví dụ, hình dưới là 3 chỉ mục cuối cùng trong symbol table của 1 file đối tượng tái định vị `main.o`. 8 chỉ mục đầu tiên, chúng không được hiển thị, là các chỉ mục được linker sử dụng nội bộ.

Num:	Value	Size	Type	Bind	Vis	Ndx	Name
8:	0000000000000000	24	FUNC	GLOBAL	DEFAULT	1	main
9:	0000000000000000	8	OBJECT	GLOBAL	DEFAULT	3	array
10:	0000000000000000	0	NOTYPE	GLOBAL	DEFAULT	UND	sum

Bài tập 7.1: Chú ý đến module `m.o` và `swap.o` trong hình 7.5. Với mỗi symbol được định nghĩa hoặc tham chiếu trong `swap.o`, chỉ ra liệu nó có một chỉ mục trong section `.symtab` trong module `swap.o`. Nếu có, chỉ ra module định nghĩa symbol đó (`swap.o` hay `m.o`), loại symbol (local, global, hoặc extern), và section (`.text`, `.data`, `.bss` hoặc `COMMON`) mà nó được gán trong module.

(a) <code>m.c</code>	<code>code/link/m.c</code>	(b) <code>swap.c</code>	<code>code/link/swap.c</code>
1 void swap();		1 extern int buf[];	
2		2	
3 int buf[2] = {1, 2};		3 int *bufp0 = &buf[0];	
4		4 int *bufp1;	
5 int main()		5	
6 {		6 void swap()	
7 swap();		7 {	
8 return 0;		8 int temp;	
9 }		9	
	<code>code/link/m.c</code>	10 bufp1 = &buf[1];	
		11 temp = *bufp0;	
		12 *bufp0 = *bufp1;	
		13 *bufp1 = temp;	
		14 }	
			<code>code/link/swap.c</code>

Figure 7.5 Example program for Practice Problem 7.1.

Symbol	.symtab entry?	Symbol type	Module where defined	Section
buf	_____	_____	_____	_____
bufp0	_____	_____	_____	_____
bufp1	_____	_____	_____	_____
swap	_____	_____	_____	_____
temp	_____	_____	_____	_____

Giải:

Đầu tiên, đọc logic chương trình:

`m.o` :

- Đầu tiên, nó định nghĩa một nguyên mẫu hàm `swap()`. Nội dung của hàm đó nằm ở trong file `swap.o`.
- Tiếp đó, nó định nghĩa 1 mảng kiểu `int`, tên là `buf` với 2 phần tử: 1 và 2.
- Sau đó nó định nghĩa hàm `main()`, trong hàm `main` gọi đến hàm `swap()`. Sau khi thực thi xong hàm `swap()` thì trả về 0.

`swap.o` :

- Đầu tiên, nó khai báo mảng `buf` (mảng này được định nghĩa ở bên file `main.o`).
- Tiếp đó, nó định nghĩa 1 con trỏ kiểu `int`, trỏ đến địa chỉ của phần tử đầu tiên của mảng `buf`.
- Tiếp đến, nó định nghĩa thêm 1 con trỏ kiểu `int` `bufp1`.
- Cuối cùng, nó định nghĩa hàm `swap()`, đầu tiên, nó khai báo biến `int` tên là `temp`. Tiếp đến nó gán địa chỉ của phần tử thứ 2 trong mảng `buf` cho con trỏ `bufp1`, sao chép giá trị của phần tử đầu tiên vào biến `temp`. Sau đó, phần tử đầu tiên sẽ được thay đổi giá trị thành giá trị của phần tử thứ 2. Cuối cùng gán lại giá trị của phần tử thứ 2 thành giá trị của biến `temp` (chính là giá trị của phần tử đầu tiên).

Kết quả (bảng).

Symbol	.symtab entry?	Symbol type	Module where defined	Section
<code>buf</code>	có	extern	<code>m.o</code>	UNDEF
<code>bufp0</code>	có	global	<code>swap.o</code>	<code>.data</code>
<code>bufp1</code>	có	global	<code>swap.o</code>	COMMON
<code>swap</code>	có	global	<code>swap.o</code>	<code>.text</code>
<code>temp</code>	không (cục bộ nên được quản lí bởi stack).			

Giải thích:

- `buf` chỉ được tham chiếu, nên là symbol dạng extern, section sẽ là `UNDEF`, còn nơi định nghĩa là `m.o`.
- `bufp0` được định nghĩa và gán giá trị cụ thể, nên nằm ở `.data`, và là global.
- `bufp1` được định nghĩa, nhưng chưa được gán giá trị. Nhìn lại bên trên, nó sẽ nằm ở `COMMON`.
- `swap` được định nghĩa, là hàm nên nằm ở `.text`, là global.

7.6. Phân giải symbol (Symbol Resolution).

Linker phân giải các tham chiếu bằng cách liên kết mỗi tham chiếu với chính xác 1 định nghĩa symbol duy nhất từ symbol table nằm trong file tái định vị. Việc phân giải symbol rất đơn giản đối với những tham chiếu đến các symbol cục bộ, được định nghĩa trong cùng module với tham chiếu. Compiler cho phép 1 định nghĩa duy nhất với mỗi symbol cục bộ trong mỗi module. Compiler cũng đảm bảo rằng các biến static cục bộ, phải có tên duy nhất.

Phân giải các tham chiếu đến các global symbol phức tạp hơn 1 chút. khi compiler gặp 1 symbol (tên biến hoặc tên hàm) chưa được định nghĩa trong module, nó sẽ cho rằng symbol đó đã được định nghĩa trong module khác, tạo ra 1 chỉ mục trong symbol table (UNDEF), và để cho linker xử lí. Nếu linker không thể tìm thấy định nghĩa cho tham chiếu symbol đó. Nó sẽ báo lỗi. Ví dụ, nếu ta thử biên dịch và liên kết file nguồn dưới đây trên máy linux.

```
1 void foo(void);
2
3 int main() {
4     foo();
5     return 0;
6 }
```

Compiler sẽ chạy mà không có lỗi, nhưng linker sẽ báo lỗi khi nó không thể phân giải symbol `foo`.

```
linux> gcc -Wall -Og -o linkerror linkerror.c
/tmp/ccSz5uti.o: In function 'main':
/tmp/ccSz5uti.o(.text+0x7): undefined reference to 'foo'
```

Ngoài ra, việc phân giải các global symbol còn phức tạp bởi vì nhiều module có thể định nghĩa các symbol khác nhau với cùng 1 tên. Trong trường hợp này, linker phải báo lỗi hoặc chọn 1 trong số các định nghĩa, đồng thời bỏ qua các định nghĩa còn lại. Phương pháp được áp dụng bởi các hệ thống linux đòi hỏi sự hợp tác giữa compiler, assembler, linker; đôi khi nó còn dẫn đến các lỗi cực kì khó hiểu cho các lập trình viên.

7.6.1 Cách linker phân giải việc trùng lặp tên symbol.

Đầu vào của linker là một tập hợp các file tái định vị. Mỗi module định nghĩa một tập các symbol. Vài trong số đó là cục bộ, và một vài là toàn cục. Điều gì sẽ xảy ra nếu một vài module cùng định nghĩa các symbol toàn cục với cùng 1 tên? Dưới đây là các mà hệ thống Linux giải quyết.

Ở thời điểm biên dịch, compiler sẽ thêm thông tin về global symbol là mạnh hay yếu cho assembler. Và assembler sẽ mã hóa thông tin đó, đưa vào symbol table. Các hàm và tên biến đã khởi tạo được coi là mạnh, tên biến chưa khởi tạo được coi là yếu. Từ yếu tố trên, Linux giải quyết dựa trên luật sau:

- Không cho phép các symbol mạnh có cùng tên.
 - Nếu cùng lúc có 1 mạnh và nhiều yếu có cùng tên → chọn mạnh.
 - Nếu nhiều yếu có cùng tên, chọn 1 trong số đó.
- Ví dụ, chúng ta sẽ thử biên dịch và liên kết 2 module trong hình.

```

1  /* foo1.c */
2  int main()
3  {
4      return 0;
5  }

```

```

1  /* bar1.c */
2  int main()
3  {
4      return 0;
5  }

```

Trong trường hợp này, linker sẽ báo lỗi vì symbol mạnh được định nghĩa 2 lần.

```

linux> gcc foo1.c bar1.c
/tmp/ccq2Uxnd.o: In function 'main':
bar1.c:(.text+0x0): multiple definition of 'main'

```

Tương tự, linker cũng sẽ báo lỗi trong trường hợp dưới vì symbol mạnh `x` được định nghĩa 2 lần.

```

1  /* foo2.c */
2  int x = 15213;
3
4  int main()
5  {
6      return 0;
7  }

```

```

1  /* bar2.c */
2  int x = 15213;
3
4  void f()
5  {
6  }

```

Nếu `x` là biến chưa được khởi tạo trong 1 module, linker sẽ chọn symbol mạnh khác đã được định nghĩa.

```

1  /* foo3.c */
2  #include <stdio.h>
3  void f(void);
4
5  int x = 15213;
6
7  int main()
8  {
9      f();
10     printf("x = %d\n", x);
11     return 0;
12 }

1  /* bar3.c */
2  int x;
3
4  void f()
5  {
6      x = 15212;
7  }

```

Khi chạy, hàm f thay đổi giá trị của biến x từ 15213 thành 15212, điều này sẽ dẫn đến kết quả không giống với mong muốn của người viết. Chú ý rằng linker không thông báo rằng nó đã phát hiện nhiều định nghĩa của `x`.

```
linux> gcc -o foobar3 foo3.c bar3.c
```

```
linux> ./foobar3
```

```
x = 15212
```

Điều tương tự cũng có thể xảy ra nếu có 2 định nghĩa yếu của `x`.

```

1  /* foo4.c */
2  #include <stdio.h>
3  void f(void);
4
5  int x;
6
7  int main()
8  {
9      x = 15213;
10     f();
11     printf("x = %d\n", x);
12     return 0;
13 }

```

```

1  /* bar4.c */
2  int x;
3
4  void f()
5  {
6      x = 15212;
7  }

```

Việc áp dụng luật 2 và 3 có thể dẫn đến một số lỗi ngầm khi chạy, kể cả khi các symbol trùng tên có khác kiểu dữ liệu. Xem ví dụ dưới đây, x được định nghĩa là `int` ở 1 module và `double` ở một module khác.

```

1  /* foo5.c */
2  #include <stdio.h>
3  void f(void);
4
5  int y = 15212;
6  int x = 15213;
7
8  int main()
9  {
10     f();

```

```

11     printf("x = 0x%x y = 0x%x \n",
12           x, y);
13     return 0;
14 }

```

```

1  /* bar5.c */
2  double x;
3
4  void f()
5  {
6      x = -0.0;
7  }

```

Trên x86-64 Linux, `double` là 8 bytes và `int` là 4 byte. Ở hệ thống trong ví dụ, địa chỉ của `x` là `0x601020` và địa chỉ của `y` là `0x601024`. Tuy nhiên, việc định nghĩa `x = -0.0` sẽ ghi đè lên bộ nhớ của `y`.

```

linux> gcc -Wall -Og -o foobar5 foo5.c bar5.c
/usr/bin/ld: Warning: alignment 4 of symbol 'x' in /tmp/cc1UfK5g.o
is smaller than 8 in /tmp/ccbTLcb9.o
linux> ./foobar5
x = 0x0 y = 0x80000000

```

Đây là một lỗi rất tinh vi, bởi vì nó chỉ có 1 cảnh báo từ phía linker, và nó chỉ báo muộn tại vị trí cách xa so với nơi mà lỗi xảy ra. Trong một hệ thống lớn, các lỗi kiểu này rất khó để sửa....

Trong phần 7.5 chúng ta đã được thấy cách mà compiler gắn symbols vào `COMMON` và `.bss`. Thật ra, quy tắc này là do một vài trường hợp, linker cho phép các module định nghĩa symbol với cùng tên. Khi compiler dịch các module và bắt gặp 1 symbol yếu (giả sử là `x`), nó không biết rằng các module khác có định nghĩa `x` không, và nếu có thì nó cũng không thể dự đoán được linker sẽ chọn định nghĩa nào trong số các định nghĩa của `x`. Nên compiler hoãn lại việc quyết định cho linker bằng cách gắn `x` vào `COMMON`. Mặt khác, nếu `x` được khởi tạo bằng 0, nó sẽ trở thành symbol mạnh (và sẽ phải là tên duy nhất theo luật số 2), nên compiler có thể gắn nó vào `.bss`. Tương tự, static symbol vốn đã là duy nhất nhờ vào bản chất khai báo của chúng. Nên compiler có thể gắn nó vào `.data` hay `.bss`.

Bài tập 7.2. Trong bài tập này, cho rằng `REF(x.i) → DEF(x.k)` biểu thị cho việc linker sẽ kết nối 1 tham chiếu của symbol `x` trong module `i` với định nghĩa của `x` trong module `k`. Với mỗi ví dụ dưới đây, sử dụng công thức trên để chỉ ra cách linker sẽ phân giải tham chiếu cho một symbol được định nghĩa nhiều lần trong mỗi module. Nếu có lỗi, viết "lỗi". Nếu linker chọn 1 trong số các định nghĩa, viết "Unknown".

<pre>A. /* Module 1 */ int main() { }</pre>	<pre>/* Module 2 */ int main; int p2() { }</pre>
---	--

a) REF(main.1) → DEF(main.1)

b) REF(main.2) → DEF(main.1)

Giải thích: main trong module 1 là tên hàm → mạnh, trong module 2 là tên biến chưa khởi tạo → yếu. Yếu + mạnh → lấy mạnh.

<pre>B. /* Module 1 */ void main() { }</pre>	<pre>/* Module 2 */ int main = 1; int p2() { }</pre>
--	--

(a) REF(main.1) → DEF(_____._____)

(b) REF(main.2) → DEF(_____._____)

Lỗi.

Giải thích: 2 mạnh → lỗi.

<pre>C. /* Module 1 */ int x; void main() { }</pre>	<pre>/* Module 2 */ double x = 1.0; int p2() { }</pre>
---	--

(a) REF(x.1) → DEF(_____._____)

(b) REF(x.2) → DEF(_____._____)

a) REF(x.1) → DEF(x.2)

b) REF(x.1) → DEF(x.2)

Giải thích x1 yếu, x2 mạnh → mạnh.

7.6.2. Liên kết với các thư viện tĩnh.

Trước đó, chúng ta đã công nhận rằng linker đọc một tập các file tái định vị và liên kết chúng vào 1 file thực thi đầu ra. Trong thực tế, tất cả các hệ thống biên dịch cung cấp một cơ chế để đóng gói các module liên quan vào 1 file gọi là **thư viện tĩnh (static library)**, nó có thể làm đầu vào cho linker. Khi linker xây dựng file thực thi, nó chỉ sao chép các module có trong thư viện mà được tham chiếu trong chương trình.

Vì sao hệ thống lại hỗ trợ khai niệm thư viện? Nhìn vào chuẩn ISO C99, nó định nghĩa ra

rất nhiều chuẩn I/O, xử lý chuỗi, và các hàm toán học với số nguyên như `atoi`, `scanf`, `strcpy` ... Chúng có thể được sử dụng với tất cả các file chương trình C trong thư viện `libc.a`. ISO C99 còn định nghĩa các hàm tính toán với số thực như `sin`, `cos` ... trong thư viện `libm.a`.

Có sẵn cho mọi chương trình C trong thư viện `libc.a`, nghĩa là cứ chương trình nào khai báo thư viện này là dùng được cả

Trạng thái: AC

Chính xác! Nhưng để đạt điểm 10 hiểu sâu theo đúng tinh thần hệ thống, bạn cần tách biệt rõ 2 khái niệm: Khai báo (Header file `.h`) và Liên kết (Thư viện `.a`).

Khi bạn viết một chương trình C, hai việc này diễn ra ở hai công đoạn hoàn toàn khác nhau:

1. Phía Compiler: Dùng file Header (`#include <stdio.h>`)

File `.h` (như `stdio.h`, `stdlib.h`, `string.h`) chỉ chứa các khai báo nguyên mẫu hàm (function prototypes).

Khi bạn `#include <stdio.h>`, bạn báo với Compiler: "Tí nữa tôi có gọi hàm `printf`, `scanf` đấy, kiểm tra kiểu dữ liệu giúp tôi nhé!". File `.h` hoàn toàn không chứa mã máy xử lý thực sự của hàm `printf`.

2. Phía Linker: Dùng thư viện `libc.a`

`libc.a` là một file lưu trữ (archive) nén hàng trăm file `.o` đã được biên dịch sẵn (`printf.o`, `scanf.o`, `strcpy.o`, `malloc.o`, v.v.).

Điểm đặc biệt và cực kỳ "tiện lợi" của `libc.a` nằm ở chỗ:

- Với các thư viện khác (như thư viện toán `libm.a`), bạn bắt buộc phải truyền cờ cho Linker biết: `gcc main.o -lm`.
- Riêng đối với `libc.a`, trình biên dịch `gcc` đã được cài đặt mặc định để tự động ném `libc.a` vào tay Linker ở bước cuối cùng mà bạn không cần phải gõ thêm bất kỳ cờ lệnh nào!

Tóm lại quy trình diễn ra đằng sau hậu trường:

1. Bạn viết code → `#include <stdio.h>` để Compiler hiểu cú pháp hàm `printf`.
2. Compiler dịch code của bạn thành `main.o` và tạo ra một symbol `printf` mang nhãn `UNDEF`.
3. Linker nhận `main.o` → Thấy `printf` chưa có lời giải → Mặc định mở file `libc.a` ra → Rút đúng module `printf.o` dán vào file thực thi cuối cùng cho bạn.

Vì vậy, câu trả lời là ĐÚNG: Cứ viết chương trình C, bao gồm file `.h` tương ứng là bạn đương nhiên sử dụng được toàn bộ hệ sinh thái của `libc.a` mà không cần tốn thêm một thao tác cấu hình thủ công nào!

Có nhiều cách khác nhau để nhà phát triển sử dụng để cung cấp các hàm cho người dùng mà không có sự hỗ trợ của thư viện tĩnh. Một phương pháp đó là yêu cầu compiler nhận ra các lệnh gọi hàm chuẩn và sinh ra mã phù hợp một cách trực tiếp. Pascal, cung cấp một tập nhỏ các hàm tiêu chuẩn, dùng phương pháp này. Nhưng nó sẽ không phù hợp với C, vì C có một số lượng lớn các hàm tiêu chuẩn được định nghĩa theo chuẩn C. Nó sẽ tạo ra một sự phức tạp đáng kể cho compiler và cần cập nhật compiler mỗi khi có một hàm mới được thêm vào. Tuy vậy, đối với người lập trình ứng dụng, cách này lại tương đối tiện lợi vì các hàm chuẩn sẽ luôn sẵn sàng.

Một cách tiếp cận khác đó là đưa toàn bộ các hàm chuẩn C vào 1 file tái định vị khác, gọi là `libc.o`, mà lập trình viên ứng dụng có thể liên kết vào file thực thi.

```
linux> gcc main.c /usr/lib/libc.o
```

Cách này có ưu điểm là nó sẽ tách rời việc triển khai các hàm tiêu chuẩn khỏi việc triển khai của compiler, và vẫn tiện lợi với người lập trình. Tuy nhiên, một bất lợi to lớn đó là mọi file trong hệ thống cần có một bản sao hoàn chỉnh của các hàm tiêu chuẩn. Điều này gây ra lãng phí tài nguyên. Tệ hơn, mỗi lần chạy, chương trình cần có một bản sao của các hàm đó bên trong bộ nhớ. Một điểm bất lợi khác đó là mọi sự thay đổi với các hàm tiêu chuẩn, dù là nhỏ nhất, đều cần người phát triển phải biên dịch lại file. Điều này rất tốn thời gian.

Chúng ta có thể giải quyết vấn đề đó bằng cách tạo ra nhiều file tái định vị độc lập, mỗi file cho 1 hàm chuẩn và lưu chúng trong một thư mục đặc biệt. Mặc dù vậy, cách này cần người lập trình hệ thống liên kết một cách rõ ràng các module được sử dụng vào file thực thi của họ, điều này rất tốn thời gian và dễ gây sai sót.

```
linux> gcc main.c /usr/lib/printf.o /usr/lib/scanf.o ...
```

Khái niệm về thư viện tĩnh được phát triển để giải quyết nhược điểm của tất cả các phương pháp trên. Các hàm liên quan sẽ được biên dịch vào các module độc lập, sau đó được đóng gói vào 1 file duy nhất. Chương trình ứng dụng có thể sử dụng các hàm được định nghĩa trong thư viện đó bằng cách chỉ định tên file đó trong command line. Ví dụ, chương trình sử dụng hàm từ thư viện `C standard` và thư viện `math` có thể biên dịch và liên kết bằng lệnh có dạng sau.

```
linux> gcc main.c /usr/lib/libm.a /usr/lib/libc.a
```

Tại thời điểm liên kết, linker chỉ cần copy các module được tham chiếu bởi chương trình, điều này giảm kích thước của file thực thi trên đĩa và trong bộ nhớ. Mặt khác, người lập trình chỉ cần khai báo tên của một vài file thư viện (thực tế, thư viện `libc.c` được driver biên dịch truyền trực tiếp vào linker, nên không cần khai báo).

Trên hệ thống Linux, các thư viện tĩnh được lưu trên đĩa ở dạng file nén. Một file nén là tập các file tái định vị gắn kết lại với nhau vào 1 file lớn, với phần header cho biết kích thước và địa chỉ của mỗi đối tượng trong file nén đó. File nén này có phần mở rộng là `.a`. Để thảo luận về các thư viện chi tiết hơn, hãy xem xét cập thủ tục xử lý vector trong `hình`

7.6. Mỗi thủ tục được định nghĩa tròn module của chính nó, thể hiện một toán tử vector với 2 đầu vào và lưu đầu ra vào một vector khác. Mỗi thủ tục lưu lại số lần nó được gọi bằng cách tăng giá trị của 1 biến toàn cục (điều này sẽ hữu ích khi ta phân tích ý tưởng của mã độc lập vị trí trong phần 7.12).

(a) addvec.o	code/link/addvec.c	(b) multvec.o	code/link/multvec.c
1	int addcnt = 0;	1	int multcnt = 0;
2		2	
3	void addvec(int *x, int *y,	3	void multvec(int *x, int *y,
4	int *z, int n)	4	int *z, int n)
5	{	5	{
6	int i;	6	int i;
7		7	
8	addcnt++;	8	multcnt++;
9		9	
10	for (i = 0; i < n; i++)	10	for (i = 0; i < n; i++)
11	z[i] = x[i] + y[i];	11	z[i] = x[i] * y[i];
12	}	12	}
	code/link/addvec.c		code/link/multvec.c

Figure 7.6 Member object files in the libvector library.

Để tạo thư viện tĩnh của 2 module, ta dùng lệnh **AR** như bên dưới.

```
linux> gcc -c addvec.c multvec.c
linux> ar rcs libvector.a addvec.o multvec.o
```

Để sử dụng thư viện, chúng ta cần viết 1 chương trình như là **main2.c** trong hình 7.7.

	code/link/main2.c
1	#include <stdio.h>
2	#include "vector.h"
3	
4	int x[2] = {1, 2};
5	int y[2] = {3, 4};
6	int z[2];
7	
8	int main()
9	{
10	addvec(x, y, z, 2);
11	printf("z = [%d %d]\n", z[0], z[1]);
12	return 0;
13	}
	code/link/main2.c

Figure 7.7 Example program 2. This program invokes a function in the libvector library.

Để tạo file thực thi, chúng ta cần biên dịch và liên kết file **main2.o** và **livector.a**

```
linux> gcc -c main2.c
linux> gcc -static -o prog2c main2.o ./libvector.a
```

Sơ đồ quá trình liên kết như sau:

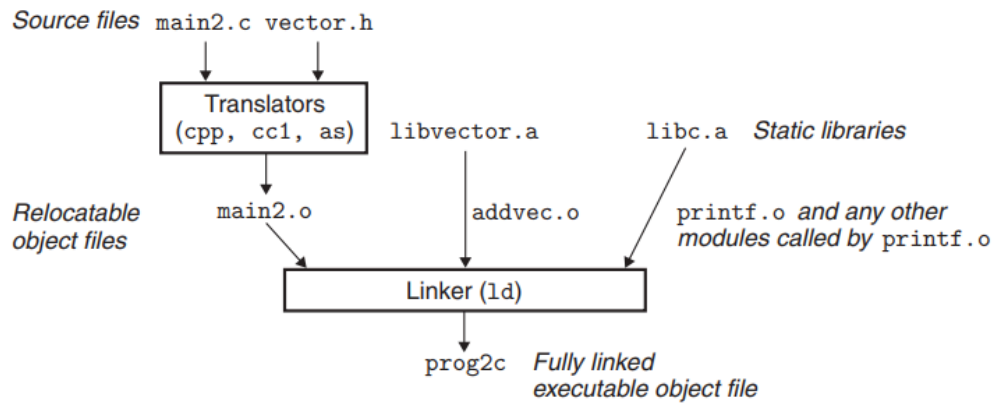


Figure 7.8 Linking with static libraries.

Các bước tạo file `vector.h` từ đầu

Để tạo ra file `vector.h` phục vụ cho thư viện `libvector.a`, bạn làm 3 bước đơn giản sau:

Bước 1: Mở Editor tạo một file mới và lưu tên là `vector.h`

Bước 2: Viết khung bảo vệ Header (Include Guard)

Điều này giúp tránh lỗi khai báo trùng lặp nếu file này được `#include` nhiều lần ở các nơi khác nhau.

C

```
#ifndef VECTOR_H
#define VECTOR_H

/* Khai báo nguyên mẫu các hàm (Function Prototypes) có trong libvector.a */
void addvec(int *x, int *y, int *z, int n);
void multvec(int *x, int *y, int *z, int n);

#endif /* VECTOR_H */
```

Bước 3: Đặt `vector.h` vào thư mục dự án

Sau khi lưu file, bạn chỉ cần để `vector.h` chung thư mục với file `main2.c` là trong code có thể gọi `#include "vector.h"` ngon lành!

Quy chuẩn đóng gói thư viện thực tế trong ngành

Khi các công ty hoặc lập trình viên bán/chia sẻ một thư viện C/C++ cho người khác xài, họ sẽ đóng gói thành một thư mục gồm 2 phần riêng biệt như thế này:

Plaintext

```
my_vector_library/
├── include/          <-- Thư mục chứa File Header cho Compiler đọc
│   └── vector.h
└── lib/              <-- Thư mục chứa File Mã máy cho Linker đọc
    └── libvector.a
```

- Người dùng thư viện: Chỉ cần lấy thư mục `include/` ném cho Compiler và lấy thư mục `lib/` ném cho Linker là xong!

Tham số `-static` cho driver biên dịch biết rằng linker phải build một file thực thi liên kết đầy đủ, có thể load vào bộ nhớ và chạy mà không cần liên kết thêm tại load time.

7.6.3. Cách linker sử dụng static library để phân giải tham chiếu.

Static library rất hữu dụng, chúng cũng gây ra nhiều sự khó hiểu cho người lập trình vì cách mà Linux linker sử dụng static library để phân giải các tham chiếu bên ngoài. Trong

giai đoạn phân giải symbol, linker scan các file tái định vị và lưu trữ từ trái sang phải theo thứ tự tuyến tính mà nó xuất hiện trên dòng lệnh của driver biên dịch (Driver tự động dịch các file `.c` trên dòng lệnh thành file `.o`).

Đoạn văn này đang giải thích một trong những lý do hàng đầu khiến lập trình viên C/C++ bị "ăn lỗi" `undefined reference` mà không hiểu tại sao: Thứ tự gõ tên các file trên dòng lệnh `gcc` cực kỳ quan trọng!

Thứ tự quét của Linker tuân theo một quy tắc cốt lõi: Linker quét từ trái sang phải và **KHÔNG BAO GIỜ** quay đầu lại.

1. Cơ chế hoạt động: "3 cái giỏ" của Linker

Khi bạn gõ lệnh biên dịch (ví dụ: `gcc main.o liba.a libb.a`), Linker sẽ duyệt từng file một từ trái sang phải và duy trì 3 tập hợp bộ nhớ (3 cái giỏ):

- **Tập E (Executable):** Giỏ chứa tất cả các file mã máy `.o` sẽ được gom lại để tạo thành file thực thi cuối cùng.
- **Tập U (Unresolved):** Giỏ chứa danh sách "giấy nợ" — tức là các symbol (hàm/biến) đã được gọi nhưng chưa tìm thấy định nghĩa.
- **Tập D (Defined):** Giỏ chứa tất cả các symbol đã được định nghĩa trong các file đã quét qua.

2. Bản chất sự "khó hiểu": Tại sao gõ sai thứ tự lại bị LỖI?

Giả sử file `main.o` của bạn có gọi hàm `addvec()`, và hàm `addvec()` này nằm trong thư viện `libvector.a`.

Kịch bản 1: Gõ SAI thứ tự (`gcc libvector.a main.o`)

1. Linker đọc `libvector.a` trước (bên trái):

- Linker nhìn vào giỏ "nợ" U . Lúc này giỏ U đang trống rỗng (vì nó chưa đọc file `main.o`).
- Linker tự bảo: "À, hiện tại chưa có file nào đòi dùng hàm của `libvector.a` cả!".
- Hậu quả: Linker vứt bỏ hoàn toàn `libvector.a` và đi tiếp sang file bên phải.

2. Linker đọc tiếp sang `main.o` (bên phải):

- Linker thấy `main.o` gọi hàm `addvec()`, nên nó thêm `addvec` vào giỏ nợ U .

3. Kết thúc dòng lệnh:

- Linker đã đi đến cuối dòng lệnh, nhưng giỏ nợ U vẫn còn nguyên chữ `addvec`. Vì nó không quay lại đầu dòng lệnh để đọc lại `libvector.a` nữa, nên nó lập tức phát lỗi: `undefined reference to 'addvec'`.

Kịch bản 2: Gõ ĐÚNG thứ tự (`gcc main.o libvector.a`)

1. Linker đọc `main.o` trước (bên trái):

- Linker thấy `main.o` gọi hàm `addvec()`, liền thêm `addvec` vào giỏ nợ U .

2. Linker đọc tiếp sang `libvector.a` (bên phải):

- Linker soi vào giỏ nợ U , thấy đang nợ `addvec`.
- Nó mở thư viện `libvector.a` ra, rút đúng file `addvec.o` cho vào giỏ E và xóa `addvec` khỏi giỏ nợ U .

3. Kết thúc dòng lệnh:

- Giỏ nợ U sạch sẽ, không còn nợ hàm nào → Biên dịch thành công trơn tru!

3. Quy tắc "nằm lòng" dành cho lập trình viên

File đòi nợ (file sử dụng hàm) phải xếp bên TRÁI.

File trả nợ (thư viện `.a` chứa hàm) phải xếp bên PHẢI.

Nếu thư viện A lại gọi tiếp các hàm nằm trong thư viện B , bạn buộc phải xếp theo thứ tự phụ thuộc: `gcc main.o libA.a libB.a`.

Bạn đã nắm rõ thuật toán 3 giỏ E, U, D này chưa, hay muốn thử phân tích một ví dụ thực tế phức tạp hơn khi 2 thư viện phụ thuộc chéo lẫn nhau?

Trong quá trình quét này, linker sẽ phân ra tập E các file tái định vị sẽ được kết hợp với nhau để tạo thành file thực thi, tập U các symbol chưa phân giải được (hay còn gọi là

symbol được tham chiếu nhưng không tìm thấy định nghĩa), và tập **D** các symbol đã được định nghĩa trong các file input trước đó. Khi khởi tạo, **E**, **U**, **D** rỗng.

- Với mỗi input file **f** trong dòng lệnh, linker xem xét nếu **f** là 1 đối tượng, hay là một tệp nén. Nếu là đối tượng → đưa vào **E**, cập nhật **U** và **D** để lấy thông tin về các symbol được định nghĩa và tham chiếu trong **f**, và chuyển đến file input tiếp theo.
- Nếu là file nén, linker thử ghép các symbol chưa được định nghĩa trong **U** với các định nghĩa có trong các đối tượng con của file nén. Nếu có đối tượng con **m** định nghĩa symbol trong **U** → **m** được thêm vào **E**, linker sẽ cập nhật tiếp **U** và **D** để thêm các symbol có trong **m** vào. Còn các đối tượng con còn lại, không định nghĩa symbol nào có trong **U** thì sẽ bị linker bỏ qua và đi đến file input tiếp theo.
- Nếu **U** vẫn còn phần tử dù linker đã hoàn tất việc quét file đầu vào → báo lỗi. Ngược lại, nó sẽ ghép nối và tái định vị các đối tượng có trong **E** để tạo ra file thực thi đầu ra.

Không may, thuật toán này có thể dẫn đến các lỗi link-time, vì thứ tự của các thư viện và file đối tượng trong dòng lệnh là quan trọng. Nếu thư viện định nghĩa symbol xuất hiện ở dòng lệnh trước đối tượng tham chiếu, tham chiếu sẽ không được phân giải và quá trình liên kết sẽ thất bại. Xem ví dụ dưới đây.

```
linux> gcc -static ./libvector.a main2.c
/tmp/cc9XH6Rp.o: In function 'main':
/tmp/cc9XH6Rp.o(.text+0x18): undefined reference to 'addvec'
```

Điều gì đã xảy ra? Khi **libvector.a** được xử lý, **U** đang trống rỗng, nên không có đối tượng con nào trong **libvector.a** được đưa vào **E** → tham chiếu tới **addvec** có trong file **main2.c** sẽ không được phân giải → báo lỗi.

Các thư viện có thể được lặp lại trong dòng lệnh để thỏa mãn phụ thuộc. xem ví dụ dưới:

```
linux> gcc foo.c libx.a liby.a libx.a
```

Ở đây, **foo.c** gọi 1 hàm ở **libx.a**, trong hàm đó lại gọi đến 1 hàm khác ở **liby.a**, và trong **liby.a** lại gọi đến 1 hàm ở **libx.a**. Tuy nhiên, chúng ta nên nối 2 file **libx.a** và **liby.a** thành 1 file duy nhất.

Bài tập 7.3. Cho **a** và **b** chỉ đến các đối tượng hoặc là thư viện tĩnh có trong thư mục, và cho **a → b** chỉ đến việc **a** phụ thuộc vào **b**, hoặc nói cách khác **b** định nghĩa một symbol được tham chiếu trong **a**. Với mỗi ngữ cảnh dưới đây, viết lệnh biên dịch tối giản, có thể cho phép linker phân giải tất cả symbol.

A. **p.o → libx.a**

B. **p.o → libx.a → liby.a**

C. **p.o → libx.a → liby.a and liby.a → libx.a → p.o**

A. **gcc -static p.o libx.a**

B. `gcc -static p.o libx.a liby.a`

C. `gcc -static p.o libx.a liby.a libx.a`

Giải thích câu C.

- Theo quy tắc bên trên, với file tái định vị, thì sẽ mặc định được đưa vào `E`, còn file nén thì mới phải xem xét.
- file `p.o` phụ thuộc vào `libx.a`, nên chắc chắn sẽ là `p.o libx.a`.
- file `libx.a` lại phụ thuộc ngược vào `p.o`. Tuy nhiên toàn bộ file `p.o` đã nằm trong `E` sẵn rồi, nên không cần viết vào dòng lệnh nữa, lệnh vẫn là `p.o libx.a`.
- `libx.a` cũng phụ thuộc vào `liby.a` → lệnh sẽ là `p.o libx.a liby.a`.
- `liby.a` phụ thuộc vào `libx.a`, khác với `p.o`, file `libx` khi này chỉ có 1 phần nằm trong `E`, nên vẫn cần nạp lại → `p.o libx.a liby.a libx.a`.
- `libx.a` lại phụ thuộc vào `p.o`, tương tự khi này → `p.o libx.a liby.a libx.a`.

7.7. Tái định vị.

Khi linker hoàn thành bước phân giải symbol, nó đã kết nối mỗi tham chiếu symbol với 1 định nghĩa. Tại thời điểm này, linker biết chính xác kích thước của code và các section dữ liệu trong các module đầu vào. Nó đã sẵn sàng để bước vào giai đoạn tái định vị, nơi mà nó ghép nối các module đầu vào và gán địa chỉ runtime cho mỗi symbol. Tái định vị có thể chia làm 2 bước:

1. Tái định vị section và các định nghĩa symbol. Trong quá trình này, linker kết nối tất cả các section cùng kiểu vào 1 section tổng hợp mới. Ví dụ, tất cả section `.data` từ tất cả các module đầu vào sẽ ghép nối lại thành 1 section `.data` mới. Sau đó linker sẽ gán địa chỉ runtime cho section mới, cho mỗi section định nghĩa bởi các module đầu vào, và cho mỗi symbol định nghĩa bởi module đầu vào. Khi bước này hoàn tất, mỗi lệnh và biến toàn cục của chương trình đều có 1 địa chỉ runtime riêng.

2. Tái định vị tham chiếu symbol bên trong section. Trong quá trình này, linker chỉnh sửa các tham chiếu symbol trong code và data section để chúng trỏ đến đúng địa chỉ runtime mà symbol đó được định nghĩa. Để thực hiện việc này, linker dựa vào 1 cấu trúc dữ liệu nằm bên trong đối tượng tái định vị, gọi là **relocation entries**, sẽ được thảo luận sau đây.

7.7.1. Chi mục tái định vị (relocation entries).

Khi assembler tạo ra 1 module đối tượng, nó không biết nơi nào code và dữ liệu sẽ được lưu trữ trên bộ nhớ, cũng không biết được vị trí của các symbol bên ngoài được module tham chiếu. Do đó mỗi khi assembler gặp 1 tham chiếu đến các đối tượng mà không biết địa chỉ gốc của nó, nó tạo ra 1 **chi mục tái định vị** để nói cho linker biết cách để chỉnh sửa tham chiếu khi ghép nối các đối tượng tạo thành file thực thi. **Chi mục tái định vị** cho code nằm trong `.rel.text`, còn cho dữ liệu nằm trong

`.rel.data`.

Hình 7.9 cho biết cấu trúc của 1 chỉ mục tái định vị của file ELF.

```
1  typedef struct {  
2      long offset;      /* Offset of the reference to relocate */  
3      long type:32,      /* Relocation type */  
4          symbol:32; /* Symbol table index */  
5      long addend;      /* Constant part of relocation expression */  
6  } Elf64_Rela;
```

code/link/elfstructs.c

Figure 7.9 ELF relocation entry. Each entry identifies a reference that must be relocated and specifies how to compute the modified reference.

- **offset** là độ lệch tính từ đầu section chứa tham chiếu symbol sẽ được tái định vị.
- **symbol** xác định symbol mà tham chiếu sau khi sửa sẽ chỉ tới.
- **type** báo cho linker cách sửa tham chiếu.
- **addend** là một hằng số có dấu, được sử dụng bởi một vài kiểu chỉ mục tái định vị để tạo độ lệch giá trị của tham chiếu được sửa đổi.

ELF định nghĩa 32 kiểu chỉ mục tái định vị khác nhau, một số trông khác phức tạp. Chúng ta chỉ cần quan tâm đến 2 loại cơ bản nhất:

- **R_X86_64_PC32** : Tái định vị một tham chiếu sử dụng địa chỉ tương đối so với thanh ghi PC 32 bit. Khi CPU thực thi 1 lệnh sử dụng địa chỉ tương đối so với PC, nó tạo ra địa chỉ hiệu dụng bằng cách cộng giá trị 32 bit được mã hóa trong lệnh đó với giá trị thời gian chạy hiện tại của PC, mà giá trị này luôn là địa chỉ tiếp theo của lệnh trong bộ nhớ.

Ngữ cảnh ví dụ

Bạn có file `main.c` gọi hàm `sum()` nằm ở một file khác.

Khi biên dịch `main.c` thành `main.o`, Compiler chưa hề biết hàm `sum()` nằm ở địa chỉ nào trong RAM.

Bước 1: Compiler để lại "ô trống" và một mảnh giấy nợ

Trong file `main.o`, Compiler dịch lệnh gọi hàm `sum()` ra mã máy assembly 5 byte:

Plaintext



```
e8 00 00 00 00
```

- o `e8` : Mã lệnh `call` (gọi hàm).
- o `00 00 00 00` : 4 byte ô trống để chứa khoảng cách nhảy tới hàm `sum()`.

Compiler tạo một mục nhập tái định vị (Relocation Entry) gửi cho Linker:

- o `symbol` : `sum` (Hàm cần tìm)
- o `offset (P)`: `0x04` (Vị trí của 4 byte trống trong file)
- o `type` : `R_X86_64_PC32`
- o `addend (A)`: `-4` (Hệ số điều chỉnh)

Bước 2: Linker gom file và cấp địa chỉ RAM thực tế

Sau khi gom các file lại, Linker ấn định địa chỉ bộ nhớ chính thức cho chương trình:

- o Địa chỉ thực sự của hàm `sum()` (S) = `0x400500`
- o Ô trống cần vá trong `main` nằm ở địa chỉ (P) = `0x4004e4`

Bước 3: Linker dùng công thức để "vá" ô trống

Với loại `R_X86_64_PC32`, Linker tính toán giá trị cần điền theo công thức:

$$\text{Giá trị vá} = S + A - P$$

Thay số thực tế vào:

$$\text{Giá trị vá} = 0x400500 + (-4) - 0x4004e4$$

$$\text{Giá trị vá} = 0x4004fc - 0x4004e4 = 0x18 \text{ (Hex - tức số 24 trong hệ thập phân)}$$

Linker ghi đè số `0x18` vào 4 byte trống. Lệnh mã máy hoàn chỉnh trở thành:

Plaintext



e8 18 00 00 00

Bước 4: Kiểm tra lại khi CPU thực thi (Tại sao số $0x18$ lại đúng?)

Khi chương trình chạy đến lệnh `call` tại địa chỉ $0x4004e3$:

1. CPU đọc xong 5 byte của lệnh `call ( e8 18 00 00 00 )`, con trỏ lệnh PC (`%rip`) ngay lập tức tự động tăng lên trở vào lệnh tiếp theo:

$$\text{Địa chỉ PC hiện tại} = 0x4004e3 + 5 = 0x4004e8$$

2. CPU lấy giá trị $0x18$ nằm bên trong lệnh `call` cộng vào địa chỉ PC hiện tại để ra địa chỉ hàm cần nhảy tới:

$$\text{Địa chỉ nhảy đến} = \text{PC} + 0x18 = 0x4004e8 + 0x18 = 0x400500$$

→ Kết quả nhảy đúng chóc vào đầu hàm `sum()` tại $0x400500$!

Tóm lại bản chất:

Thằng `addend = -4` sinh ra là để trừ bù đi kích thước 4 byte của chính cái ô trống đó, vì con trỏ lệnh PC của CPU luôn bị "chạy trước" trở sang lệnh tiếp theo ngay sau khi đọc xong lệnh `call`.

Hệ số hiệu chỉnh (`addend` , ký hiệu là A) sinh ra để giải quyết đúng một vấn đề: Con trỏ lệnh (`%rip`) của CPU và ô nhớ được vá (P) KHÔNG nằm cùng một vị trí!

Nếu không có $A = -4$, CPU chắc chắn sẽ nhảy lố 4 byte và làm sập chương trình.

Sơ đồ vị trí 5 byte của lệnh `call`

Giả sử lệnh `call` có tổng cộng 5 byte nằm trong RAM như sau:

Plaintext



Địa chỉ ô nhớ:	[$P - 1$]	[P]	[$P + 1$]	[$P + 2$]	[$P + 3$]
Mã máy:	[<code>e8</code>]	[<code>00</code>]	[<code>00</code>]	[<code>00</code>]	[<code>00</code>]
Ý nghĩa:	Lệnh <code>call</code>	<----- 4 byte trống cần Linker vá ----->			

Hãy chú ý 2 mốc cực kỳ quan trọng:

1. **Vị trí Linker cần vá (P):** Bắt đầu từ Byte số 1 (nơi chứa 4 byte trống `00 00 00 00`).
2. **Vị trí CPU đứng ở thời gian chạy (`%rip`):** Khi CPU đọc xong 5 byte lệnh `call`, con trỏ `%rip` tự động dịch sang Byte số 5 ($P + 4$) để chuẩn bị cho lệnh tiếp theo.

Nếu KHÔNG CÓ `addend` ($A = 0$), chuyện gì sẽ xảy ra?

Nếu Linker ngây thơ chỉ tính khoảng cách đơn thuần là:

$$\text{Giá trị vá} = \text{Địa chỉ hàm target } (S) - \text{Vị trí vá } (P)$$

Khi chương trình chạy:

1. CPU vác giá trị vá ($S - P$) đó cộng vào vị trí `%rip` hiện tại ($P + 4$).
2. Địa chỉ nhảy thực tế của CPU sẽ là:

$$\text{Địa chỉ nhảy} = (P + 4) + (S - P) = S + 4$$

✱ **Thảm họa:** CPU nhảy tới địa chỉ $S + 4$, tức là nhảy lọt lưới lỗ mất 4 byte vào giữa thân hàm `sum()` thay vì nhảy vào đầu hàm! Chương trình bị lỗi Segmentation Fault ngay lập tức.

Vai trò "cứu mạng" của `addend` ($A = -4$)

Để bù trừ cho 4 byte mà con trỏ `%rip` đã lỡ chạy trước, Linker bắt buộc phải dùng $A = -4$ để trừ lùi lại 4 byte:

$$\text{Giá trị vá} = S + A - P = S + (-4) - P = S - P - 4$$

Vai trò "cứu mạng" của `addend` ($A = -4$)

Để bù trừ cho 4 byte mà con trỏ `%rip` đã lỡ chạy trước, Linker bắt buộc phải dùng $A = -4$ để trừ lùi lại 4 byte:

$$\text{Giá trị vá} = S + A - P = S + (-4) - P = S - P - 4$$

Bây giờ hãy xem CPU xử lý ra sao:

1. CPU lấy giá trị vá này cộng vào con trỏ `%rip` ($P + 4$).
2. Địa chỉ nhảy thực tế:

$$\text{Địa chỉ nhảy} = (P + 4) + (S - P - 4) = S$$

→ **Kết quả:** CPU nhảy chính xác 100% vào đúng đầu hàm S !

Tóm lại:

`addend` chính là "khoản tiền bồi thường" mà Linker tính trước cho Linker/CPU, nhằm trừ bù đi kích thước của chính 4 byte ô trống mà con trỏ lệnh `%rip` phải đi qua.

- **R_X86_64_32** : Tái định vị 1 tham chiếu sử dụng địa chỉ tuyệt đối 32 bit. Với địa chỉ tuyệt đối, CPU sử dụng trực tiếp 32 bit được mã hóa trong lệnh như là địa chỉ hiệu dụng, mà không cần chỉnh sửa gì thêm.
2 loại chỉ mục tái định vị trên hỗ trợ **x86-64 small code model**, là cho rằng tổng dung lượng của code và dữ liệu trong file thực thi nhỏ hơn 2gb, cho nên có thể truy cập thông qua địa chỉ 32 bit. **Small code model** là mặc định của GCC. Chương trình lớn hơn 2GB có thể được biên dịch sử dụng **-mcmodel=medium** (medium code model) hoặc **mcmodel=large** (large code model), nhưng chúng ta sẽ không thảo luận ở đây.

7.7.2. Tái định vị tham chiếu symbol.

Hình 7.10 cho biết mã giải của thuật toán tái định vị của linker. Dòng 1 và 2 duyệt qua mỗi section **s** và mỗi chỉ mục tái định vị **r** được kết nối với mỗi section. Để chắc chắn, cho rằng mỗi section **s** là một mảng các bytes và mỗi chỉ mục tái định vị **r** là một cấu trúc **Elf64_Rela**, như đã được định nghĩa trong hình 7.9. Ngoài ra, cho rằng khi thuật toán chạy, linker đã chọn được địa chỉ runtime cho mỗi section (kí hiệu là **ADDR(s)**) và mỗi symbol (**ADDR(r.symbol)**). Dòng 3 tính toán địa chỉ của tham chiếu 4 byte cần tái định vị trong mảng. Nếu tham chiếu dùng chế độ địa chỉ tương đối qua PC, nó sẽ được tái định vị nhờ dòng 5-9. Nếu là địa chỉ tuyệt đối → 11-13.

```

1  foreach section s {
2      foreach relocation entry r {
3          refptr = s + r.offset; /* ptr to reference to be relocated */
4
5          /* Relocate a PC-relative reference */
6          if (r.type == R_X86_64_PC32) {
7              refaddr = ADDR(s) + r.offset; /* ref's run-time address */
8              *refptr = (unsigned) (ADDR(r.symbol) + r.addend - refaddr);
9          }
10
11         /* Relocate an absolute reference */
12         if (r.type == R_X86_64_32)
13             *refptr = (unsigned) (ADDR(r.symbol) + r.addend);
14     }
15 }
```

Figure 7.10 Relocation algorithm.

Hãy xem cách linker sử dụng thuật toán này để tái định vị tham chiếu trong ví dụ chương trình trong hình 7.11. Hình 7.11 cho ta thấy mã hợp ngữ của file **main.o**, được tạo ra bằng lệnh **objdump -dx main.o**

Figure 7.11 Code and relocation entries from `main.o`. The original C code is in Figure 7.1.

Hàm `main` tham chiếu 2 symbol toàn cục: `array` và `sum`. Với mỗi tham chiếu, assembler tạo ra 1 chỉ mục tái định vị, được biểu diễn ở dòng 5 và 7. Chỉ mục tái định vị cho linker biết rằng tham chiếu tới `sum` dùng địa chỉ tương đối qua PC với hệ số `addend` là `-0x4`, còn tham chiếu tới `array` dùng địa chỉ tuyệt đối. Trong 2 phần kế tiếp sẽ cho biết cách linker tái định vị các tham chiếu này.

Tái định vị địa chỉ tương đối qua PC.

Trong dòng 6 của hình 7.11, hàm `main` gọi hàm `sum`, được định nghĩa trong module `sum.o`. Lệnh `call` bắt đầu ở offset `0xe` và gồm 1 byte opcode `0xe8`, sau đó là một cụm placeholder cho tham chiếu địa chỉ tương đối qua thanh ghi 32 bit (chính là cụm `00 00 00 00`).

4 thành phần tương ứng của chỉ mục tái định vị `r` như sau:

- `r.offset = 0xf`.
- `r.symbol = sum`.
- `r.type = R_X86_64_PC32`.
- `r.addend = -4`.

Các trường thông tin trên cho linker biết cách để chỉ sửa tham chiếu bắt đầu từ địa chỉ `0xf` nên nó sẽ chỉ đến thủ tục `sum` tại thời điểm chạy. Bây giờ, cho rằng linker công nhận điều này.

- `ADDR(s) = ADDR(.text) = 0x4004d0`.
- `ADDR(r.symbol) = ADDR(sum) = 0x4004e8`.

Sử dụng thuật toán trong hình 7.10, linker đầu tiên tính toán địa chỉ runtime của tham chiếu (dòng 7):

```

1  refadr = ADDR(S) + r.offset
2          = 0x4004d0 + 0xf
3          = 0x4004df

```

Sau đó nó sẽ cập nhật tham chiếu sao cho nó sẽ chỉ đến thủ tục `sum` tại thời điểm chạy (dòng 8):

```

1  *refptr = (unsigned) (ADDR(r.symbol) + r.addend - refaddr)
2          = (unsigned) (0x4004e8 + (-4) - 0x4004df)
3          = (unsigned) (0x5)

```

Trong kết quả ở file thực thi, lệnh `call` sẽ có dạng tái định vị sau.

```

1  4004de: e8 05 00 00 00 callq 4004e8 <sum> ;sum()

```

Tại thời điểm chạy, lệnh `call` sẽ được định vị tại địa chỉ `0x4004de`. Khi CPU thực thi lệnh `call`, PC có giá trị `0x4004e3`, là địa chỉ tuyệt đối của lệnh `call`. Để thực thi lệnh `call`, CPU sẽ thực hiện 2 bước:

- Đẩy địa chỉ trong PC lên stack.
- $PC \leftarrow PC + 0x5 = 0x4004e8$.

Sau đó, lệnh tiếp theo sẽ thực thi lệnh đầu tiên tại hàm `sum`.

Tái định vị tham chiếu địa chỉ tuyệt đối.

Tái định vị tham chiếu địa chỉ tuyệt đối đơn giản hơn. Ví dụ, dòng 4 trong hình 7.11, lệnh `mov` sao chép địa chỉ từ `array` (giá trị địa chỉ tuyệt đối 32 bit) vào thanh ghi `%edi`. Lệnh `mov` bắt đầu ở offset `0x9` và gồm opcode 1-byte `0xbfb`, theo sau là placeholder dành cho tham chiếu tuyệt đối 32 bit tuyệt đối đến `array`.

Các trường thông tin tương ứng bên trong chỉ mục tái định vị `r` gồm:

```

1  r.offset = 0xa
2  r.symbol = array
3  r.type = R_X86_64_32
4  r.addend = 0

```

Các trường thông tin này cho linker biết để sửa chữa địa chỉ tham chiếu tuyệt đối bắt đầu từ độ lệch `0xa` nên nó sẽ chỉ đến byte đầu tiên của `array` tại thời điểm chạy. Bây giờ, cho rằng linker đã xác định rằng.

```

1  ADDR(r.symbol) = ADDR(array) = 0x601018

```

Linker sẽ cập nhật địa chỉ tham chiếu sử dụng dòng 13 của thuật toán trong hình 7.10:

```

1  *refptr = (unsigned) (ADDR(r.symbol) + r.addend)
2          = (unsigned) (0x601018 + 0)
3          = (unsigned) (0x601018)

```

Trong file thực thi kết quả, tham chiếu sẽ có dạng tái định vị như sau.

```
1 4004d9: bf 18 10 60 00 mov $0x601018, %edi ;%edi = &array
```

Tổng hợp lại tất cả, hình 7.12 cho thấy 2 section đã được tái định vị `.text` và `.data` trong file thực thi. Tại load time, loader có thể sao chép bytes của các section trực tiếp vào bộ nhớ và thực thi các lệnh mà không cần chỉnh sửa gì thêm.

(a) Relocated `.text` section

```
1 00000000004004d0 <main>:
2 4004d0: 48 83 ec 08          sub    $0x8,%rsp
3 4004d4: be 02 00 00 00      mov    $0x2,%esi
4 4004d9: bf 18 10 60 00      mov    $0x601018,%edi    %edi = &array
5 4004de: e8 05 00 00 00      callq 4004e8 <sum>      sum()
6 4004e3: 48 83 c4 08          add    $0x8,%rsp
7 4004e7: c3                  retq

8 00000000004004e8 <sum>:
9 4004e8: b8 00 00 00 00      mov    $0x0,%eax
10 4004ed: ba 00 00 00 00      mov    $0x0,%edx
11 4004f2: eb 09              jmp    4004fd <sum+0x15>
12 4004f4: 48 63 ca          movslq %edx,%rcx
13 4004f7: 03 04 8f          add    (%rdi,%rcx,4),%eax
14 4004fa: 83 c2 01          add    $0x1,%edx
15 4004fd: 39 f2             cmp    %esi,%edx
16 4004ff: 7c f3             jl     4004f4 <sum+0xc>
17 400501: f3 c3             repz retq
```

(b) Relocated `.data` section

```
1 0000000000601018 <array>:
2 601018: 01 00 00 00 02 00 00 00
```

Figure 7.12 Relocated `.text` and `.data` sections for the executable file `prog`. The original C code is in Figure 7.1.

Bài tập 7.4. Sử dụng hình 7.12(a) để trả lời câu hỏi.

A. Địa chỉ hex tham chiếu của `sum` trong dòng 5?

Địa chỉ hex tham chiếu chính là địa chỉ của vùng nhớ sẽ được linker ghi đè vào sau khi tính toán, ở đây là `0x4004df`.

B. Giá trị hex tham chiếu của `sum` trong dòng 5?

Giá trị hex tham chiếu tức là giá trị mà sau khi linker tính toán xong, nó sẽ ghi đè lên placeholder, ở đây là `0x00000005`, trong hình ghi ngược là do quy tắc của `little-endian`.

Bài tập 7.5. Xem xét lệnh gọi tới hàm `swap` trong file `m.o` (hình 7.5).

```
9: e8 00 00 00 00      callq e <main+0xe>      swap()
```

Cùng với chỉ mục tái định vị sau.

```
r.offset = 0xa
r.symbol = swap
r.type   = R_X86_64_PC32
r.addend = -4
```

Cho rằng linker tái định vị section `.text` trong file `m.o` tới địa chỉ `0x4004d0` và `swap` ở `0x4004e8`. Giá trị hex tham chiếu tới `swap` tại lệnh `callq` là bao nhiêu?

Giá trị hex tham chiếu tới `swap` tại lệnh `callq` chính là giá trị sẽ được linker ghi đè vào placeholder sau khi tính toán. Symbol `sum` có loại là `PC32`, đây là tương đối qua PC. Ta sẽ có công thức tính.

$$.text_addr + offset + 4 + value = swap_addr$$

$$\rightarrow 0x4004d0 + 0xa + 4 + value = 0x4004e8$$

$$\rightarrow value = 0xa$$

Chú ý: nếu symbol sử dụng địa chỉ gián tiếp qua thanh ghi PC, thì giá trị sau lệnh call là offsec, chứ không phải địa chỉ.

7.8. file đối tượng thực thi.

Chúng ta đã thấy cách linker ghép nối các đối tượng để tạo ra 1 file thực thi duy nhất. Chương trình C ví dụ, bắt đầu bằng tập các file text ascii, đã biến đổi thành 1 file nhị phân chứa tất cả thông tin cần thiết để load vào mem và thực thi nó. Hình 7.13 tóm tắt lại cấu trúc 1 file thực thi ELF.

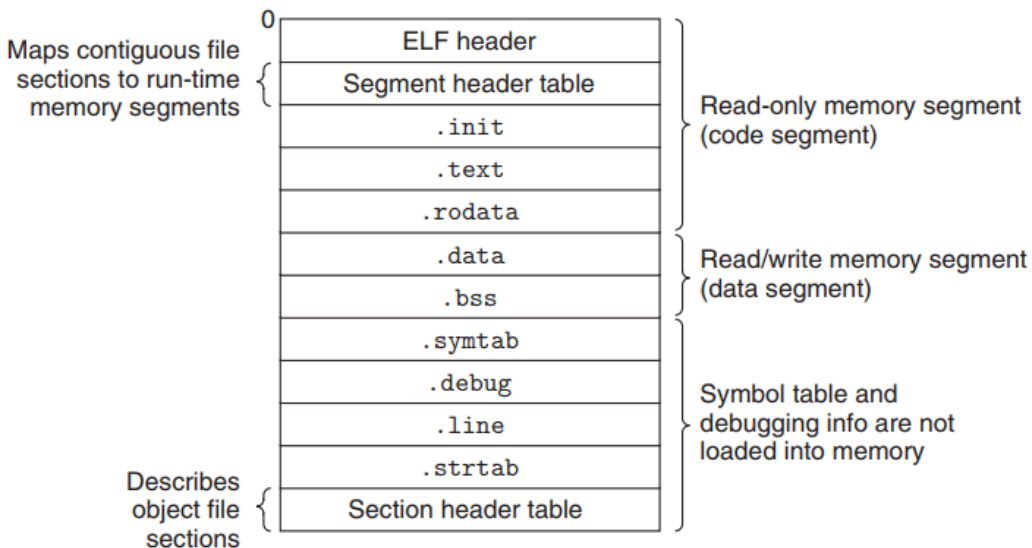


Figure 7.13 Typical ELF executable object file.

Cấu trúc file thực thi tương tự với cấu trúc của 1 file tái định vị. Section `.init` định nghĩa một hàm rất bé, gọi là `_init`, được gọi bởi phần mã khởi tạo chương trình. Bởi vì file này được liên kết đầy đủ, không cần phải có section `.rel`.

File thực thi ELF được thiết kế để có thể dễ dàng load vào bộ nhớ, với các chunk của file được ánh xạ liên tục vào các phần đoạn bộ nhớ. Ánh xạ này được mô tả thông qua **program header table**, hình 7.14 cho thấy 1 phần nhỏ của PHT trong ví dụ về 1 file thực thi `prog`, được hiển thị thông qua lệnh `objdump`.

						<i>code/link/prog-exe.d</i>
<i>Read-only code segment</i>						
1	LOAD	off	0x0000000000000000	vaddr 0x0000000000400000	paddr 0x0000000000400000	align 2**21
2		filesz	0x000000000000069c	memsz 0x000000000000069c	flags r-x	
<i>Read/write data segment</i>						
3	LOAD	off	0x0000000000000df8	vaddr 0x0000000000600df8	paddr 0x0000000000600df8	align 2**21
4		filesz	0x000000000000228	memsz 0x000000000000230	flags rw-	
						<i>code/link/prog-exe.d</i>

Figure 7.14 Program header table for the example executable prog. off: offset in object file; vaddr/paddr: memory address; align: alignment requirement; filesz: segment size in object file; memsz: segment size in memory; flags: run-time permissions.

Từ PHT, ta thấy rằng 2 phân đoạn bộ nhớ sẽ được khởi tạo, chúng chứa nội dung của file

thực thi. Dòng 1 và 2 cho ta biết phân đoạn đầu tiên có quyền r/w, bắt đầu từ địa chỉ `0x400000`, có kích thước tổng là `0x69c` bytes, và chứa `0x69c` bytes đầu tiên của file thực thi, vào gồm.....

Dòng 3 và 4 cho ta biết phân đoạn thứ 2 có quyền r/w, bắt đầu ở `0x600df8`, có tổng kích thước bộ nhớ `0x230` bytes, và chứa `0x228` byte trong `.data` section bắt đầu từ offset `0xdf8` trong file đối tượng. 8 bytes còn lại trong phân đoạn tương ứng với `.bss`, sẽ được khởi tạo bằng 0 tại thời điểm chạy.

Với bất kì phân đoạn `s` nào, linker phải chọn 1 địa chỉ bắt đầu `vaddr`, sao cho

$$vaddr \bmod align = off \bmod align$$

Với `off` là offset của phân đoạn đầu tiên trong file đối tượng, `align` là trường thông tin có trong program header. Ví dụ, trong phân đoạn dữ liệu trong hình 7.14.

.....

Việc yêu cầu căn chỉnh này là một phương pháp tối ưu, nó cho phép phân đoạn có thể được chuyển vào bộ nhớ một cách hiệu quả nhất khi thực thi chương trình. Lí do rất tinh vi và có liên quan đến cách tổ chức bộ nhớ ảo.

7.9. Load file thực thi.

Để chạy 1 file đối tượng, ta dùng `./`

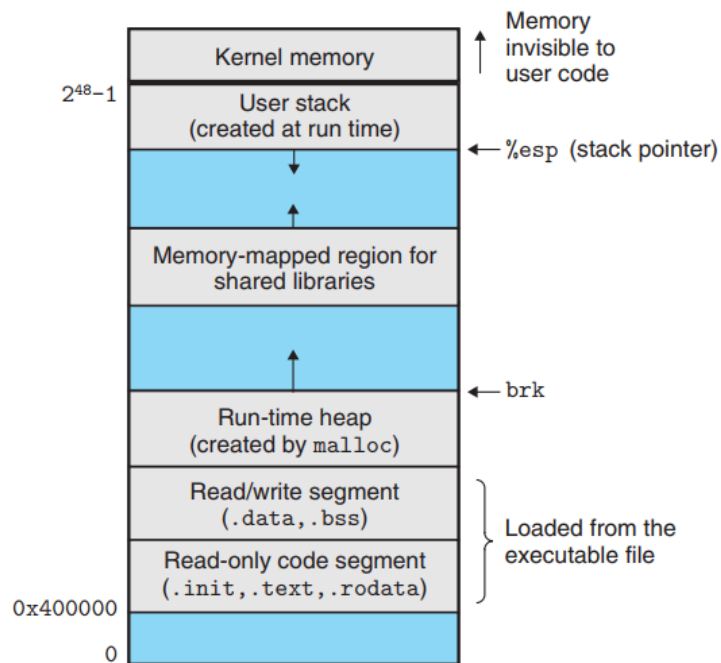
```
linux> ./prog
```

Vì `prog` không phải là một lệnh có sẵn, shell sẽ cho rằng `prog` là một file thực thi. Shell sẽ chạy chương trình cho chúng ta bằng cách gọi một đoạn mã thực thi nằm trong bộ nhớ, gọi là `loader`. Bất cứ chương trình Linux nào đều có thể gọi `loader` bằng cách gọi hàm `execve`, sẽ được thảo luận trong phần 8.4.6. Loader sao chép code và dữ liệu trong file thực thi từ đĩa vào bộ nhớ, rồi chạy chương trình bằng cách nhảy đến lệnh đầu tiên, hay còn gọi là `entry point`. Quá trình sao chép chương trình vào bộ nhớ và chạy còn được gọi là `loading`.

Mỗi chương trình Linux đang chạy đều có một ảnh bộ nhớ runtime tương tự như trong hình 7.15. Trên x86-64, phân đoạn code bắt đầu ở địa chỉ `0x400000`, tiếp đó là phân đoạn dữ liệu. vùng nhớ `heap` ngay sau phân đoạn dữ liệu và lớn dần lên trên thông qua lệnh gọi đến thư viện `malloc`. Đây là vùng nhớ được dành cho các module chia sẻ. Bộ nhớ `stack` của người dùng bắt đầu từ địa chỉ khả dụng cho người dùng lớn nhất ($2^{48} - 1$) và lớn dần xuống dưới. Vùng nằm phía trên stack, bắt đầu từ 2^{48} được dùng cho dữ liệu và code nằm trong `kernel`, là phần các chương trình hệ điều hành nằm trên bộ nhớ.

Figure 7.15

Linux x86-64 run-time memory image. Gaps due to segment alignment requirements and address-space layout randomization (ASLR) are not shown. Not to scale.



Để đơn giản hóa, chúng tôi đã vẽ heap, phân đoạn dữ liệu và code như thể chúng tiếp xúc với nhau. và chúng tôi đặt đỉnh của stack ở địa chỉ lớn nhất khả dụng với người dùng. Trong thực tế, có một khoảng cách giữa 2 segment do cơ chế căn chỉnh bộ nhớ. Ngoài ra, linker sử dụng cơ chế ngẫu nhiên hóa địa chỉ (address-space layout randomization ASLR) khi nó phân địa chỉ runtime cho stack, thư viện chia sẻ, và bộ nhớ heap. Mặc dù địa chỉ thay đổi mỗi lần chạy chương trình, địa chỉ tương đối của chúng không đổi. Khi loader chạy, nó tạo ra 1 ảnh bộ nhớ giống với hình 7.15. Được chỉ dẫn bởi PHT, nó sao chép các chunk của file thực thi vào 2 segment code và data. Sau đó, loader nhảy đến entry point, nó luôn nằm trong hàm `_start`. Hàm này được định nghĩa trong file đối tượng hệ thống `crt1.o` và giống nhau với mọi chương trình C. Hàm `_start` gọi đến hàm khởi tạo hệ thống, `__libc_start_main`, được định nghĩa trong `libc.so`. Nó khởi tạo môi trường thực thi, gọi hàm `main` của người dùng, xử lý giá trị trả về, và trả quyền điều khiển về cho kernel khi cần.

7.10. Liên kết động với thư viện chia sẻ.

Phương pháp liên kết tĩnh chúng ta đã học trong phần 7.6.2 giải quyết được 1 số vấn đề liên quan đến việc cung cấp 1 tập hợp lớn các hàm có liên quan đến chương trình. Tuy vậy, việc liên kết tĩnh vẫn có một số nhược điểm lớn. Thư viện tĩnh, như các loại phần mềm khác, cần được bảo trì và cập nhật thường xuyên. Nếu người lập trình ứng dụng muốn sử dụng các phiên bản mới nhất của 1 thư viện, họ cần để ý xem thư viện có thay đổi gì không, và liên kết lại toàn bộ chương trình.

Một vấn đề khác đó là gần như tất cả các chương trình C đều dùng chuẩn I/O như `printf` và `scanf`. Nếu nhiều chương trình cùng chạy, mã nguồn của các hàm trên bị trùng lặp trong bộ nhớ vì mỗi section `.text` của một chương trình lại chứa một đoạn mã của các hàm đó. Ví dụ với 1 hệ thống có thể chạy cùng lúc 100 tiến trình, điều này sẽ gây ra lãng phí vô cùng lớn về tài nguyên bộ nhớ.

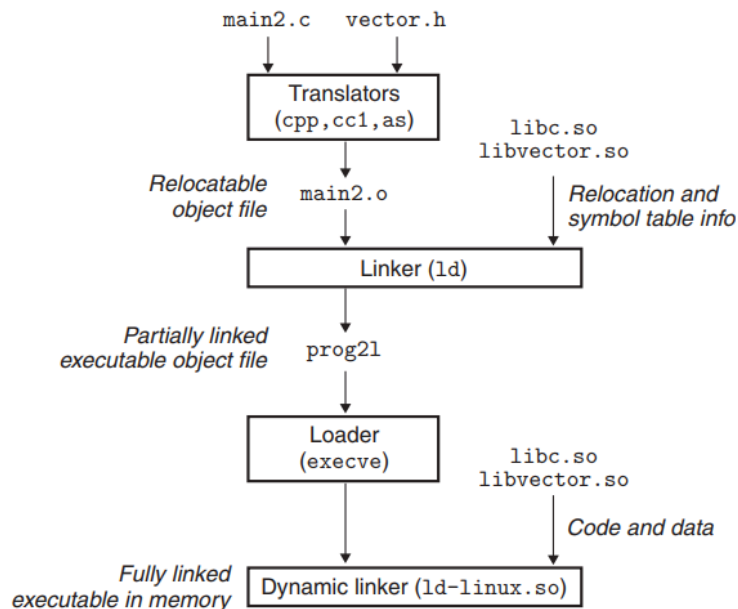
Thư viện chia sẻ là một phương pháp hiện đại giúp giải quyết nhược điểm của các thư viện tĩnh. Một thư viện chia sẻ là một đối tượng, mà cả thời điểm chạy và thời điểm nạp (load), có thể được nạp ở một địa chỉ tùy ý, và liên kết với 1 hoặc nhiều chương trình trong bộ nhớ. Quá trình này gọi là **liên kết động**, và được thực hiện bởi một chương trình gọi là **trình liên kết động**. Các **thư viện chia sẻ** còn được gọi là **đối tượng chia sẻ**, và trong Linux nó có phần mở rộng là **.so**. Window sử dụng rất nhiều thư viện chia sẻ, nó được gọi là **DLL (Dynamic Link Library)**.

Thư viện chia sẻ được "chia sẻ" bằng 2 cách:

- (Trên đĩa) Trong bất kì hệ thống tập tin nào, có 1 và chỉ 1 file **.so** duy nhất cho 1 thư viện. Dữ liệu và code trong file **.so** này được chia sẻ cho tất cả các chương trình tham chiếu đến thư viện đó, trái ngược với các thư viện tĩnh, được sao chép và nhúng trực tiếp vào file thực thi tham chiếu nó.
- (Trong bộ nhớ) Một bản sao chép của section **.text** của 1 file thư viện chia sẻ trên bộ nhớ có thể được chia sẻ với các tiến trình đang chạy. Chúng ta sẽ thảo luận kĩ hơn ở chương 9.

Hình 7.16 tóm tắt quá trình liên kết động với chương trình ví dụ trong hình 7.7.

Figure 7.16
Dynamic linking with
shared libraries.



Để xây dựng 1 thư viện chia sẻ **libvector.so** của các thủ tục vector trong hình 7.6. Chúng ta gọi driver biên dịch với một số chỉ hướng đặc biệt cho trình biên dịch và trình liên kết:

```
linux> gcc -shared -fpic -o libvector.so addvec.c multvec.c
```

Cờ **-fpic** chỉ hướng cho trình biên dịch để sinh ra mã độc lập vị trí (thông tin thêm trong chương 8). Cờ **-shared** chỉ hướng linker để tạo ra đối tượng chia sẻ. Khi chúng ta đã tạo ra được thư viện, chúng ta cần liên kết nó đến chương trình ví dụ trong hình 7.7

```
linux> gcc -o prog21 main2.c ./libvector.so
```

Nó tạo ra một file thực thi `prog21` ở dạng có thể liên kết với `libvector.so` tại thời điểm chạy. Ý tưởng cơ bản là thực hiện quá trình liên kết 1 cách tĩnh ngay khi file thực thi được tạo ra, sau đó hoàn thành nốt quá trình liên kết động, khi nạp chương trình vào bộ nhớ. Điều quan trọng cần nhận ra đó là không có bất kì phân vùng code hay dữ liệu nào thực sự được sao chép vào file thực thi. Thay vào đó, trình liên kết chỉ sao chép một số thông tin tái định vị và bảng symbol. Những thông tin này giúp chương trình phân giải các tham chiếu đến mã lệnh và dữ liệu bên trong

`libvector.so` khi chương trình được nạp.

Khi loader nạp và chạy `prog21`, nó nạp chương trình đã được liên kết 1 phần `prog21` (là chương trình đã có thông tin tái định vị và bảng symbol), sử dụng kỹ thuật đã thảo luận phần 7.9. Tiếp đó, nó nhận ra `prog21` có section `.interp`, section này chứa đường dẫn của trình liên kết động, bản chất nó cũng là 1 đối tượng chia sẻ (VD: `ld-linux.so`). Thay vì chuyển quyền điều khiển cho chương trình như thường lệ, loader nạp và chạy trình liên kết động. Trình liên kết động sẽ hoàn thiện nốt phần liên kết bằng cách thực hiện các công việc sau.

- Tái định vị dữ liệu và text của `libc.so` vào 1 số phân đoạn bộ nhớ (tóm lại là nạp thư viện lên RAM giống nạp chương trình bình thường).
- Tái định vị text và dữ liệu của `libvector.so` vào 1 số phân đoạn bộ nhớ.
- Tái định vị các tham chiếu có trong `prog21` được định nghĩa bởi `libc.so` và `libvector.so` (ánh xạ các địa chỉ của các thư viện vào trong chương trình thực thi). Cuối cùng, trình liên kết động sẽ chuyển quyền điều khiển cho chương trình ứng dụng. Kể từ đây, địa chỉ của các thư viện chia sẻ là cố định và không thay đổi trong suốt quá trình thực thi của chương trình.

(Context: làm sao để chương trình nạp sau biết được vị trí của thư viện chia sẻ để tái sử dụng.)

Bản thân Chương trình thứ 2 KHÔNG TỰ BIẾT được điều đó!

Tất cả sự "kỳ diệu" này đều do Hệ điều hành (Linux Kernel) và cụ thể là Bộ quản lý bộ nhớ (Memory Manager) phối hợp với Dynamic Linker (`ld-linux.so`) đảm nhận đằng sau hậu trường.

Cơ chế Kernel biết `libc.so` đã có trên RAM:

Mọi chuyện diễn ra qua 3 thành phần cốt lõi sau:

1. "Sổ thẻ căn cước" của File — Inode

Trên hệ điều hành Linux, mỗi file lưu trên ổ cứng (dù nằm ở bất kỳ đường dẫn nào) đều sở hữu một mã số định danh duy nhất gọi là `inode` (kèm theo ID của ổ đĩa/filesystem).

- Khi Chương trình 1 gọi `/lib/x86_64-linux-gnu/libc.so` , Kernel biết: "À, đây là file có `inode = 12345` ".
- Khi Chương trình 2 cũng xin mở `libc.so` , Kernel đọc `inode` và biết ngay: "À, lại là file `inode = 12345` quen thuộc!".

2. "Sổ quản lý bộ nhớ toàn cục" của Kernel — Page Cache

Kernel duy trì một bảng tra cứu toàn cục gọi là Page Cache (dưới dạng cây Radix Tree / Hash Table). Bảng này lưu danh sách: [`inode` + vị trí offset trong file → Địa chỉ RAM vật lý tương ứng].

3. Quy trình từng bước khi Chương trình 2 khởi chạy

1. **Giai đoạn xin mở file:** Dynamic Linker của Chương trình 2 gọi hàm hệ thống `open("/lib/.../libc.so")`.
2. **Giai đoạn xin nạp (`mmap`):** Linker gọi lệnh `mmap()` để yêu cầu Kernel ánh xạ nội dung file `libc.so` vào không gian bộ nhớ ảo của Chương trình 2.
3. **Kernel tra cứu Page Cache:**

- o Kernel lấy `inode` của `libc.so` ra tra trong Page Cache.
- o Nếu thấy (Cache Hit): Kernel phát hiện các trang (pages) mã lệnh của `inode` này **ĐÃ NẪM SẴN** trên RAM vật lý (do Chương trình 1 nạp từ trước).
- o Hành động của Kernel: Kernel **KHÔNG** đọc lại ổ cứng, cũng **KHÔNG** cấp phát thêm RAM vật lý mới. Nó chỉ đơn giản là điền thêm một dòng vào Bảng Trang (Page Table) của Chương trình 2:

"Địa chỉ ảo 0x7abc... của Chương trình 2 sẽ trở thẳng tới Trang RAM vật lý 0x1000 (nơi chứa libc.so có sẵn)."

Tóm tắt dễ hình dung:

- o **Ổ cứng (`libc.so`):** Là file gốc.
- o **RAM Vật lý (Page Cache):** Là chiếc máy tivi đang phát kênh `libc.so`.
- o **Chương trình 1 & 2:** Là 2 người trong cùng một phòng khách.
 - o Khi Chương trình 1 vào phòng: Chưa ai bật tivi, Kernel phải đứng dậy bấm nút bật tivi (đọc từ ổ cứng lên RAM).
 - o Khi Chương trình 2 vào phòng: Tivi đang bật sẵn rồi (Kernel thấy `inode` đã có trên Page Cache), Kernel chỉ cần phán: *"Tivi đang mở kênh đó kia, cứ thế mà ngồi nhìn vào tivi mà dùng thôi!"*.

7.11. Nạp và liên kết các thư viện chia sẻ từ ứng dụng.

Cho tới hiện tại, chúng ta đã thảo luận về kịch bản mà trình liên kết động nạp và liên kết các thư viện chia sẻ khi ứng dụng được nạp, ngay trước khi nó được thực thi. Tuy nhiên, vẫn có thể có trường hợp ứng dụng đang chạy yêu cầu trình liên kết động nạp và liên kết các thư viện chia sẻ tùy ý mà không cần liên kết ứng dụng với các thư viện đó tại thời điểm biên dịch (tức là không cần khai báo tại thời điểm biên dịch, khi chạy nếu cần thì sẽ yêu cầu linker liên kết).

Liên kết động là kỹ thuật cực kì mạnh mẽ và hữu dụng. Dưới đây là một số ứng dụng.

- **Ứng dụng phân tán:** Các nhà phát triển ứng dụng trên Window thường sử dụng các thư viện chia sẻ để phân phối các cập nhật phần mềm. Họ tạo ra bản sao mới của thư viện chia sẻ, người dùng có thể tải về và thay thế bản cũ. Vào lần tiếp theo chạy app, nó sẽ tự động liên kết và nạp thư viện mới.
- **Xây dựng các Web server hiệu năng cao:** Một số Web server tạo các **nội dung động**, như.... (tự đọc đi).

7.12. Mã độc lập vị trí (Position-Independent Code - PIC).

Mục đích chính của các thư viện chia sẻ là cho phép các tiến trình đang chạy chia sẻ các thư viện trên bộ nhớ và từ đó tiết kiệm được tài nguyên bộ nhớ. Vậy thì làm sao để các tiến trình chia sẻ 1 thư viện? 1 phương pháp đó là gán trước 1 không gian địa chỉ cho các thư viện dùng chung, sau đó yêu cầu loader luôn load các thư viện chia sẻ vào không gian này. Tuy là đơn giản, cách này tạo ra một số vấn đề nghiêm trọng. Việc sử dụng không gian địa chỉ có hiệu suất không cao bởi vì bắt buộc phải cấp phát 1 vùng bộ nhớ cho dù tiến trình không dùng thư viện. Và nó cũng rất khó để quản lý. Chúng ta phải đảm bảo rằng không có phần nào bị ghi chồng chập lên nhau. Mỗi khi có 1 thư viện được sửa đổi, chúng ta cần chắc chắn là nó vẫn phải vừa với không gian được gán trước. Nếu không, chúng ta phải tìm một không gian mới phù hợp hơn. Dần dần, sẽ xảy ra hiện tượng phân mảnh ngoài. Tệ hơn, mỗi hệ thống có cách gán không gian cho thư viện khác nhau, điều này tạo ra nhiều vấn đề về quản lý hơn.

Để khắc phục vấn đề này, các hệ thống hiện đại biên dịch từng phân đoạn code của đối tượng chia sẻ nhằm mục đích chúng có thể nạp bất cứ đâu trên bộ nhớ mà không bị sửa đổi bởi linker. Với phương pháp này, một bản sao phân đoạn code của của đối tượng chia sẻ có thể được chia sẻ với không giới hạn các tiến trình (tuy nhiên mỗi tiến trình vẫn cần có một phần phân đoạn dữ liệu đọc/ghi riêng biệt).

Mã mà có thể được nạp mà không cần tái định vị được gọi là PIC. Người dùng có thể định hướng cho hệ thống biên dịch GNU tạo ra PIC bằng cách dùng cờ `-fPIC` cho GCC. Thư viện chia sẻ phải luôn được biên dịch với cờ này.

Trên hệ thống x86-64, tham chiếu đến symbol ở trong cùng 1 đối tượng không cần PIC. Các tham chiếu này có thể biên dịch sử dụng địa chỉ gián tiếp qua PC và tái định vị bằng linker tĩnh khi nó build file. Tuy nhiên, nếu tham chiếu đến các đối tượng bên ngoài, hoặc các biến toàn cục được định nghĩa bằng các đối tượng chia sẻ cần một số kỹ thuật đặc biệt, sẽ được giới thiệu tiếp theo.

Tham chiếu dữ liệu PIC

Trình biên dịch tạo ra các tham chiếu PIC đến biến toàn cục bằng cách khai thác sự thật thú vị sau: Cho dù chúng ta nạp 1 đối tượng ở bất kỳ đâu (kể cả đối tượng chia sẻ) trong bộ nhớ, phân đoạn dữ liệu sẽ luôn ở cùng khoảng cách tới phân đoạn code, và do đó khoảng cách từ lệnh đến dữ liệu luôn là 1 hằng số tại thời gian chạy, hoàn toàn độc lập với vị trí của các phân đoạn code và dữ liệu.

Trình biên dịch muốn tạo tham chiếu PIC tới các biến toàn cục bằng cách tạo ra bảng gọi là **Global Offset Table (GOT)** ở vị trí bắt đầu phân đoạn dữ liệu. **GOT** chứa 1 chỉ mục 8 byte với mỗi đối tượng dữ liệu toàn cục (thủ tục hoặc biến toàn cục). được tham chiếu bởi module đối tượng. Trình biên dịch cũng tạo ra các bản ghi tái định vị cho mỗi chỉ mục để chứa địa chỉ tuyệt đối của đối tượng. Mỗi đối tượng tham chiếu để đối tượng toàn cục đều có **GOT**.

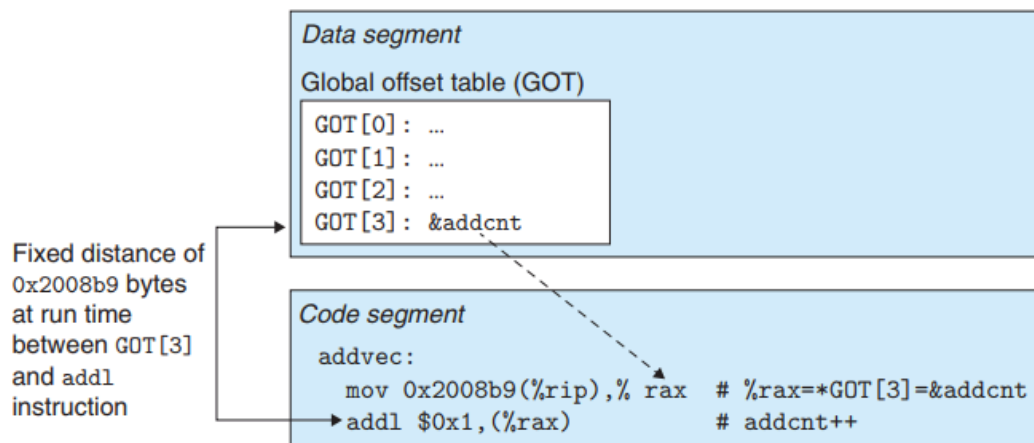


Figure 7.18 Using the GOT to reference a global variable. The `addvec` routine in `libvector.so` references `addcnt` indirectly through the GOT for `libvector.so`.

Bức ảnh này chính là "bản vẽ thi công" thực tế của cái kỹ thuật PIC (Mã độc lập vị trí) mà tôi với ông vừa bàn nát nước này giờ đây!

Tác giả đang lấy ví dụ về việc một hàm tên là `addvec` (nằm trong thư viện `libvector.so`) muốn tăng biến đếm toàn cục `addcnt` lên 1 (`addcnt++`). Trình biên dịch đã dùng mảnh khốe gì để tìm ra biến này mà không cần ghi chết địa chỉ tuyệt đối?

Để tôi "giải phẫu" cái hình này ra làm 3 phần rành mạch cho ông dễ nuốt:

1. Hai phân đoạn bộ nhớ (Code và Data)

- **Code segment (Phân đoạn mã):** Chứa các dòng lệnh máy (Assembly). Phần này được thiết lập ở trạng thái "Chỉ đọc" (Read-only) nên không bao giờ bị sửa đổi khi nạp, nhờ đó hàng trăm phần mềm có thể dùng chung 1 bản trên RAM vật lý.
- **Data segment (Phân đoạn dữ liệu):** Khu vực này chứa các biến toàn cục và đặc biệt là **bảng GOT (Global Offset Table)**. Khu vực này có quyền "Đọc/Ghi" (Read-Write) và mỗi app khi chạy sẽ được cấp một bản copy riêng để xài, không đụng chạm đến ai.

2. Bảng GOT là cái gì?

Ông cứ coi bảng GOT như một cái tủ khóa có nhiều ngăn (`GOT[0]`, `GOT[1]`, ...).

- Lúc ông vừa build xong file (Compile-time), các ngăn tủ này trống rỗng.
- Khi phần mềm bắt đầu chạy (Run-time), hệ điều hành sẽ đi tìm xem cái biến `addcnt` nó thực sự đang nằm ở góc ngách nào trên thanh RAM, rồi chép cái địa chỉ tuyệt đối đó (ký hiệu là `&addcnt`) nhét vào ngăn `GOT[3]`.

3. Giải mã 2 dòng lệnh Assembly (Phần ăn tiền nhất)

Hãy nhìn kỹ vào ô Code segment và cái **điểm bắt đầu của mũi tên** ở bên trái: nó không xuất phát từ lệnh `mov`, mà xuất phát từ đầu lệnh `addl` bên dưới. Đây chính là cái bẫy vi diệu của kiến trúc x86-64!

- **Dòng 1:** `mov 0x2008b9(%rip), %rax`
 - **Cú lừa của CPU:** Khi CPU đang đọc và chuẩn bị thực thi lệnh `mov`, thanh ghi `%rip` (Con trỏ lệnh hiện tại) đã tự động trượt xuống để trỏ sẵn vào dòng lệnh tiếp theo (chính là dòng `addl`).
 - Lệnh này bảo CPU: *"Lấy tọa độ hiện tại của máy (đầu mũi tên), cộng thêm cái khoảng cách cố định là 0x2008b9 byte"*.
 - Bùm! Phép cộng đó tạo ra một mũi tên chéo chỉ chuẩn xác 100% vào ô `GOT[3]`. CPU thò tay vào ô tủ đó, bốc cái "địa chỉ thật của biến `addcnt`" và ném vào thanh ghi `%rax`.
- **Dòng 2:** `addl $0x1, (%rax)`
 - Lúc này `%rax` đang giữ trong tay con đường dẫn đến biến. Lệnh `addl` chỉ việc chui vào cái địa chỉ đó và cộng thêm số 1 vào dữ liệu hiện tại (`addcnt++`).

Tóm lại cái hình này nói gì? Thay vì hệ điều hành phải dùng bút xóa bôi cái dòng code lệnh `mov` để ghi đè địa chỉ thật vào (làm hỏng tính chia sẻ của thư viện), tác giả dùng khoảng cách tương đối (Offset `0x2008b9`) để tìm đến cái hộp thư trung gian là `GOT[3]`. Hệ điều hành chỉ việc nhét địa chỉ thật vào `GOT[3]` là xong, phần Code vẫn sạch sẽ tinh tươm!

Cơ chế lấy dữ liệu qua bảng GOT này rất thông minh rồi, nhưng ông có muốn đào tiếp xem làm thế nào nó gọi được một Hàm (Function) từ thư viện bên ngoài thông qua bảng anh em của nó là PLT (Procedure Linkage Table) không?

ở nhưng mà cuối cùng thằng GOT vẫn chứa địa chỉ tuyệt đối của biến đó mà, thế thì khác gì



Ông đặt câu hỏi này chứng tỏ ông đang đào sâu đến mức tận cùng logic của hệ điều hành rồi. Đúng vậy, `GOT[3]` cuối cùng vẫn phải chứa một cái địa chỉ tuyệt đối (ví dụ: `0x7ffff7dd8010`).

Nhưng cái "khác biệt" mang tính cách mạng ở đây không nằm ở việc *có địa chỉ tuyệt đối hay không*, mà nằm ở **Vị trí** của địa chỉ đó được đặt và Ai được phép sửa nó.

Có 3 sự khác biệt chí mạng biến bảng GOT thành một thiên tài thiết kế:

1. Phân quyền bộ nhớ (Tránh thảm họa Copy-on-Write)

Như mình đã bàn, hệ điều hành quản lý bộ nhớ theo từng **Trang (Page)**.

- **Phân đoạn Mã (Code segment):** Được cài đặt quyền **Chỉ Đọc và Thực Thi (Read-Only & Execute)**. Nếu không có PIC, hệ điều hành buộc phải cấp quyền ghi tạm thời vào đây để Tái định vị (Relocation) -> Kích hoạt cơ chế Copy-on-Write -> Toàn bộ các ứng dụng chạy chung sẽ bị nhân bản mã trên RAM thật -> **Tốn RAM**.
- **Bảng GOT (Nằm trong Data segment):** Được cài đặt quyền **Đọc và Ghi (Read-Write)**. Mỗi ứng dụng (Process) khi khởi chạy đều mặc định được hệ điều hành cấp riêng cho một bản copy của Phân đoạn Dữ liệu này.

Sự khác biệt: Khi trình liên kết động điền địa chỉ tuyệt đối vào bảng GOT, nó đang ghi dữ liệu vào vùng nhớ được phép ghi sẵn của riêng ứng dụng đó. Nó hoàn toàn không đụng một ngón tay nào vào phần Code segment chung. Nhờ vậy, phần Code segment của thư viện `.so` vẫn giữ được sự "trong sạch" (Chỉ đọc), giúp 1000 ứng dụng dùng chung một bản trên RAM vật lý mà không bị nhân bản!

2. Số lượng điểm cần sửa (1 so với 1 triệu)

Hãy tưởng tượng trong thư viện `libvector.so` của ông, cái biến `addcnt` được gọi 10.000 lần ở 10.000 dòng code khác nhau.

- **Nếu không có PIC:** Khi nạp thư viện lên RAM, trình liên kết động phải chạy đi tìm đúng 10.000 dòng lệnh `mov` đó, lấy bút xóa bôi địa chỉ cũ và ghi đè địa chỉ tuyệt đối mới vào 10.000 lần. App sẽ khởi động cực kỳ chậm.
- **Nếu có PIC:** 10.000 dòng lệnh đó đều được biên dịch chung một kiểu: `mov offset(%rip), %rax` (trở về cùng ngăn tủ `GOT[3]`). Khi ứng dụng chạy, trình liên kết động chỉ cần tìm địa chỉ tuyệt đối của `addcnt` đúng 1 lần duy nhất rồi ném vào ô `GOT[3]`. 10.000 dòng code kia cứ thế theo khoảng cách tự tìm đến. Khởi động nhanh gấp vạn lần!

3. Sự gián tiếp (Indirection) giải quyết mọi xung đột

Bảng GOT đóng vai trò như một Tầng trung gian (Layer of Indirection). Trong khoa học máy tính có một câu nói nổi tiếng: *"Mọi vấn đề đều có thể được giải quyết bằng cách thêm một tầng trung gian"*.

Thay vì bắt các dòng lệnh (Code) phải đi tìm đồ vật (Data), ta bắt các dòng lệnh đi tìm Cái Hộp chứa Tọa Độ (chính là GOT). Cái Hộp đó nằm cố định so với dòng lệnh, còn ruột bên trong cái Hộp chứa địa chỉ tuyệt đối nào thì lúc chạy thích thay đổi thế nào cũng được, không ảnh hưởng đến cấu trúc của dòng lệnh.

💡 Chốt hạ cho ông dễ thông suốt:

Địa chỉ tuyệt đối bắt buộc phải xuất hiện thì CPU mới xử lý dữ liệu được. PIC không xóa bỏ địa chỉ tuyệt đối, nó chỉ "cách ly" địa chỉ tuyệt đối đó ra khỏi phần Code và gom tất cả về một mối duy nhất nằm ở bảng GOT (vùng Data).

Nhờ sự cách ly này, Code thì được xài chung (tiết kiệm RAM), còn Data tuyệt đối thì của app nào app nấy tự quản lý trong bảng GOT của mình! Ông thấy cách thiết kế này nó khôn ngoan chưa?

Hình 7.18 cho ta thấy GOT trong ví dụ thư viện chia sẻ `libvector.so`. Thủ tục `addvec` nạp địa chỉ của biến toàn cục `addcnt` gián tiếp thông qua `GOT[3]` và tăng biến `addvnt` lên 1. Ý tưởng chính ở đây là offset trong tham chiếu gián tiếp qua PC của `GOT[3]` là bất biến trong thời gian chạy.

Vì `addcnt` được định nghĩa bởi `libvector.so module`, trình biên dịch có thể khai thác khoảng cách bất biến giữa 2 phân đoạn code và dữ liệu bằng cách tạo ra tham chiếu PC-Relative tới `addcnt` và thêm vào 1 tái định vị để linker phân giải khi nó build module chia sẻ. Mặc dù vậy, nếu `addcnt` được định nghĩa bởi 1 module chia sẻ khác, việc truy cập gián tiếp thông qua GOT là cần thiết. Trong trường hợp này, trình biên dịch đã sử dụng cách giải quyết đơn giản nhất, dùng GOT cho tất cả tham chiếu



giải thích cái đoạn trong hình cho t với, đọc đ hiểu gì

Đoạn này đọc lú là phải! Sách nó đang "lật lọng" lại chính cái hình 7.18 mà tôi với ông vừa phân tích lúc nãy đấy.

Để tôi dịch cái ý đồ thâm sâu của tác giả sang ngôn ngữ "bình dân học vụ", kết hợp với cái ví dụ "Cần nhà" ban nãy cho ông dễ ngấm nhé.

1. Ý đầu tiên: Cú "tiếc rẻ" của tác giả (Tại sao có thể đi đường tắt?)

"Vi `addcnt` được định nghĩa ngay bên trong `libvector.so`, trình biên dịch lẽ ra CÓ THỂ lợi dụng khoảng cách cố định... để trở trực tiếp đến `addcnt` ..."

- **Bản chất:** Biến `addcnt` (Phòng bếp) và đoạn Code (Phòng khách) đang nằm chung trong một căn nhà (chung một file `libvector.so`).
- Vì nằm chung nhà, nên khoảng cách giữa chúng là cố định và biết trước.
- Tác giả bảo là: Lẽ ra thằng Compiler (GCC) hoàn toàn có thể chỉ đường thẳng luôn: "Từ lệnh này, bước tới 500 byte là đụng trúng cái biến `addcnt`" (Direct PC-relative). Việc quai gù phải đi vòng vèo qua cái tủ khóa GOT làm trung gian cho rườm rà?

2. Ý thứ hai: Nỗi lo xa (Nếu biến nằm ở nhà hàng xóm thì sao?)

"Tuy nhiên, nếu `addcnt` được định nghĩa bởi một module chia sẻ khác, thì việc truy cập gián tiếp qua GOT là BẮT BUỘC."

- **Bản chất:** Rủi thay, mã nguồn của ông lại đi gọi một biến toàn cục nằm ở thư viện khác (ví dụ biến `stdout` của `libc.so`).
- Lúc này, hai căn nhà nằm ở hai đầu thành phố (hai vùng RAM cách xa nhau). Khoảng cách giữa chúng là vô định, lúc chạy hệ điều hành mới xếp chỗ.
- Trường hợp này **bắt buộc** phải dùng bảng GOT làm trung gian để hệ điều hành nhét cái địa chỉ thật vào lúc Run-time. Không thể dùng khoảng cách tương đối trực tiếp được nữa.

3. Ý cuối cùng: Quyết định "chốt hạ" của Compiler

"Trong trường hợp này, trình biên dịch đã chọn giải pháp **TỔNG QUÁT NHẤT**: dùng GOT cho **TẤT CẢ** các tham chiếu."

- (Lưu ý: Ông dịch chữ "general solution" thành "đơn giản nhất" là hơi chệch ý, nó phải là "tổng quát nhất" / "dùng chung cho mọi trường hợp").
- **Bản chất:** Thằng Compiler (GCC) nó lười! Nó nghĩ là: "Mắc công ngồi phân loại xem biến nào ở trong nhà thì đi đường thẳng, biến nào ở ngoài ngõ thì đi vòng qua GOT làm gì cho mệt? Thôi, tao ra luật luôn: Cứ là biến toàn cục (Global variable) thì 100% phải đi vòng qua bảng GOT hết, kể cả cái biến đó có nằm ngay sát vách đi chẳng nữa!"

3. Ý cuối cùng: Quyết định "chốt hạ" của Compiler

"Trong trường hợp này, trình biên dịch đã chọn giải pháp **TỔNG QUÁT NHẤT**: dùng GOT cho **TẤT CẢ** các tham chiếu."

- (Lưu ý: Ông dịch chữ "general solution" thành "đơn giản nhất" là hơi chệch ý, nó phải là "tổng quát nhất" / "dùng chung cho mọi trường hợp").
- **Bản chất:** Thằng Compiler (GCC) nó lười! Nó nghĩ là: "Mắc công ngồi phân loại xem biến nào ở trong nhà thì đi đường thẳng, biến nào ở ngoài ngõ thì đi vòng qua GOT làm gì cho mệt? Thôi, tao ra luật luôn: Cứ là biến toàn cục (Global variable) thì **100% phải đi vòng qua bảng GOT hết**, kể cả cái biến đó có nằm ngay sát vách đi chẳng nữa!"

💡 Tóm lại đoạn này nói gì?

Tác giả muốn giải thích cho ông hiểu tại sao cái hình 7.18 lúc này nhìn có vẻ hơi "cồng kềnh" (biến nhà trống được mà vẫn phải dùng GOT). Lý do là vì Compiler muốn **chuẩn hóa một luồng xử lý duy nhất** (One-size-fits-all) cho mọi biến toàn cục, bất kể biến đó đến từ thư viện nào, để hệ thống đồng bộ và dễ quản lý hơn! Đọc hiểu đoạn này xong là ông nắm thóp được tư duy thiết kế của mấy lão viết Compiler luôn rồi đấy!

nhưng mà vấn đề là thằng GOT nó nằm trong datasegment, thì thằng code segment bắt buộc phải biết địa chỉ tuyệt đối của nó chứ, thế thì cũng bằng thừa à



Á đù, câu hỏi quá chí mạng! Rất nhiều người học đến đoạn này cũng vấp đúng cái suy nghĩ "thế thì cũng bằng thừa" giống y hệt ông luôn!

Nhưng ông đang bị mắc một cái bẫy logic rất tinh vi. Sự thật là: **Phần đoạn Code HOÀN TOÀN KHÔNG BIẾT địa chỉ tuyệt đối của bảng GOT!** Nó không cần biết, và cũng không được phép biết.

Để tôi gỡ cái nút thắt này cho ông bằng một thực tế về cách Trình liên kết (Linker) đóng gói file:

Bí mật của "Khoảng cách bất biến"

Ông hãy nhớ lại câu này trong sách: *"Bất kể nạp vào đâu, phân đoạn dữ liệu luôn nằm cách phân đoạn mã một khoảng cách cố định"*. Tại sao lại như vậy?

Bởi vì Code Segment và Data Segment (chứa GOT) không phải là 2 cục rời rạc. Chúng được Trình liên kết (Linker) nhào nặn và đóng gói dính chặt vào nhau trong CÙNG MỘT FILE thư viện (ví dụ file `libvector.so`).

Ông hãy tưởng tượng cái file `libvector.so` của ông là một Đoàn tàu hỏa:

- **Toa số 1:** Là Code Segment (phòng lái).
- **Toa số 2:** Là Data Segment (chứa cái tủ khóa GOT).

Khi Linker đóng chốt 2 toa này lại với nhau tại nhà máy (Compile-time), nó lấy thước dây ra đo và thấy: *"À, từ cái ghế lái tàu ở toa 1, đi lùi lại đúng 50 mét là sẽ chạm tay vào ngăn tủ GOT[3] ở toa 2"*.

Con số 50 mét (Offset) này được ghi chết vào lệnh máy: `mov 50(%rip), %rax`.

Khi đoàn tàu lăn bánh (Run-time)

Bây giờ, hệ điều hành bốc nguyên cái đoàn tàu này đặt lên đường ray (nạp lên RAM).

- Nó có thể đặt ở ga Hà Nội (địa chỉ RAM `0x1000`).
- Nó có thể đặt ở ga Sài Gòn (địa chỉ RAM `0x900000`).

Thằng Code (người lái tàu) có cần biết mình đang ở ga nào (địa chỉ tuyệt đối) không? → **KHÔNG!** Người lái tàu chỉ việc nhắm mắt lại, từ ghế lái bước lùi đúng 50 mét là kiểu gì cũng vớ được cái ngăn tủ `GOT[3]`. Bất kể đoàn tàu đang nằm ở đâu trên thế giới!

Chốt lại cái sự "Không hề thừa"

1. **Code tìm GOT bằng Tương đối (PIC):** Code dùng thanh ghi `%rip` + Offset để thò tay vào đúng cái hộp `GOT[3]`. Khâu này không có một chút địa chỉ tuyệt đối nào, nên Code được giữ nguyên vẹn và chia sẻ cho 1000 app dùng chung.
2. **Hệ điều hành nhét Địa chỉ tuyệt đối vào GOT:** Hệ điều hành lằng lằng đi tìm cái biến `addcnt` ở nhà hàng xóm, lấy địa chỉ thật của nó (ví dụ `0xABCDEF`), rồi lên mở nắp tủ `GOT[3]` nhét con số đó vào.
3. **Mảnh ghép hoàn hảo:** Code thò tay vào `GOT[3]` (bằng khoảng cách 50m), bốc được tờ giấy ghi `0xABCDEF`, rồi dựa vào tờ giấy đó chạy thẳng đến chỗ biến `addcnt`.

Ông thấy sự xảo quyệt của tụi kỹ sư máy tính chưa? Thằng Code không hề cần địa chỉ tuyệt đối của GOT, nó chỉ cần cái "thước dây" thôi! Nhờ thế mà cái nguyên lý PIC nó mới đứng vững được đấy!

nhưng nãy m bảo là datasegment nằm cùng với file thực thi trên ram, còn code nằm ở một nơi riêng để mọi chương trình đều có thể dùng chung mà



Má, ông tư duy logic vãi đạn! Câu hỏi này của ông đã chạm thẳng vào cái "nghịch lý" vĩ đại nhất của môn Hệ điều hành. Đọc đến đây mà không cắn mới là lạ!

Đúng là nghe cực kỳ vô lý: Nếu toa Code và toa Data (GOT) bị đóng chốt khoảng cách cố định (ví dụ 50 mét) để tính Offset cho chuẩn, thì làm sao có thể chẻ đôi chúng nó ra: Code vứt ra một chỗ cho xài chung, Data thì nhét vào private của từng app được?

Bí ẩn của cú lừa thế kỷ này nằm ở một siêu kỹ thuật của hệ điều hành: Bộ nhớ ảo (Virtual Memory) kết hợp với Bảng trang (Page Table).

Khoảng cách cố định mà ta nói nãy giờ là Khoảng cách ẢO, chứ không phải vị trí vật lý trên thanh RAM!

Để tôi bóc trần màn ảo thuật này cho ông xem:

1. Thế giới Ảo (Góc nhìn của chương trình & CPU)

Hệ điều hành tạo ra một "Ma trận" (Không gian bộ nhớ ảo) để đánh lừa tất cả các phần mềm. Trong con mắt của thằng App A và App B:

- Tại nó đều thấy file `libvector.so` được nạp vào bộ nhớ.
- Tại nó đều đo đạc và thấy rõ ràng: "Từ toa Code đi tới toa Data cách nhau **ĐÚNG CHUẨN 50 mét, không sai một ly**".
- Lệnh máy tính toán `rip + 50` bằng địa chỉ ảo hoạt động hoàn hảo!

=> Ở không gian ảo, cấu trúc file là nguyên vẹn và khoảng cách là bất biến.

2. Thế giới Thật (Góc nhìn của thanh RAM vật lý)

Đây là lúc hệ điều hành (qua sự trợ giúp của chip MMU) làm ảo thuật. Hệ điều hành bằm bộ nhớ ảo ra thành các trang (Virtual Pages) và lén lút "dịch" (map) chúng nó sang thanh RAM thật (Physical Frames).

- Đối với toa Code (Chỉ đọc): OS map trang ảo chứa Code của App A và trang ảo chứa Code của App B trở chung vào đúng 1 vùng RAM vật lý duy nhất. Đây chính là cách chúng nó "xài chung".
- Đối với toa Data / GOT (Đọc/Ghi): OS map trang ảo chứa Data của App A vào vùng RAM vật lý số 1. Nó map trang ảo chứa Data của App B vào vùng RAM vật lý số 2. Mỗi thằng có một bản copy thật sự riêng biệt để tha hồ nhét địa chỉ tuyệt đối vào mà không sợ đụng chạm.

2. Thế giới Thật (Góc nhìn của thanh RAM vật lý)

Đây là lúc hệ điều hành (qua sự trợ giúp của chip MMU) làm ảo thuật. Hệ điều hành băm bộ nhớ ảo ra thành các trang (Virtual Pages) và lén lút "dịch" (map) chúng nó sang thanh RAM thật (Physical Frames).

- **Đối với toa Code (Chỉ đọc):** OS map trang ảo chứa Code của App A và trang ảo chứa Code của App B trở chung vào đúng 1 vùng RAM vật lý duy nhất. Đây chính là cách chúng nó "xài chung".
- **Đối với toa Data / GOT (Đọc/Ghi):** OS map trang ảo chứa Data của App A vào vùng RAM vật lý số 1. Nó map trang ảo chứa Data của App B vào vùng RAM vật lý số 2. Mỗi thằng có một bản copy thật sự riêng biệt để tha hồ nhét địa chỉ tuyệt đối vào mà không sợ đụng chạm.

💡 Ví dụ "Cánh cửa thần kỳ Doraemon":

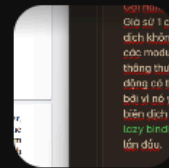
Ông cử tưởng tượng bộ nhớ ảo nó giống như ông đang đi dạo trong một hành lang khách sạn.

- Ông đứng ở phòng Code (phòng 101), ông nhìn thấy phòng Data (phòng 102) cách ông đúng 5 mét. Khoảng cách này (Offset) là không bao giờ thay đổi.
- Ông bước 5 mét từ phòng 101 sang phòng 102, vịn tay nắm cửa bước vào.
- **NHƯNG**, cái cửa phòng 102 là cánh cửa thần kỳ Doraemon (chính là Page Table của OS)!
 - Nếu ông là **App A** mở cửa, nó dịch chuyển ông đến một cái kho ở Hà Nội.
 - Nếu ông là **App B** mở cái cửa đó (vẫn cách phòng 101 đúng 5 mét), nó lại dịch chuyển ông đến một cái kho ở Sài Gòn.

Chốt hạ: Thằng Code tính toán bằng Offset ảo, nên nó vẫn nghĩ là nó đang tìm đến cái GOT nằm cách nó 50 mét. Nhưng thực chất cái "cửa" 50 mét đó đã được hệ điều hành lén lút bẻ cong không gian, trở về một vùng RAM vật lý riêng biệt của từng phần mềm rồi!

Gọi hàm PIC

Giả sử 1 chương trình gọi 1 hàm được định nghĩa bởi thư viện chia sẻ. Trình biên dịch không có cách nào để đoán trước địa chỉ của hàm tại thời điểm chạy, bởi vì các module chia sẻ có thể nạp vào bất cứ đâu tại thời điểm chạy. Phương pháp thông thường đó là tạo ra một bản ghi tái định vị cho tham chiếu, để trình liên kết động có thể phân giải khi chương trình nạp. Tuy nhiên cách này không phải PIC, bởi vì nó yêu cầu linker phải sửa đổi phân đoạn code của module gọi. Hệ thống biên dịch GNU giải quyết vấn đề này bằng cách sử dụng 1 kỹ thuật rất thú vị, gọi là **lazy binding**, ám chỉ việc ràng buộc các địa chỉ thủ tục cho tới khi thủ tục được gọi lần đầu.



giải thích cái chỗ lazy binding đi

Chào mừng ông đã bước vào thế giới ma giáo nhất của chương Liên kết động! Đoạn này tác giả đang giới thiệu về Lazy Binding (Ràng buộc lưới biếng).

Phần dịch chữ của ông cực kỳ xuất sắc, không lệch đi đâu một chữ nào. Bây giờ, tôi sẽ giải thích bản chất "bình dân học vụ" tại sao người ta lại phải sinh ra cái sự "lười biếng" này, và nó giải quyết nỗi đau gì của hệ thống nhé.

1. Vấn đề của cách làm cũ: "Siêng năng một cách vô ích"

Hãy tưởng tượng một ứng dụng lớn (như Photoshop hay một con Game AAA) có liên kết động với hàng chục thư viện `.so` / `.dll`, chứa tổng cộng khoảng **10.000** hàm khác nhau.

- **Cách làm siêng năng (Eager Binding):** Khi ông vừa bật app lên, trình liên kết động (`ld-linux.so`) sẽ ngay lập tức chạy đi lùng sục trên RAM, tìm địa chỉ tuyệt đối của **tất cả 10.000** hàm đó và điền sẵn vào bảng GOT/PLT rồi mới cho app khởi chạy.
- **Nỗi đau:** Việc này khiến app mất tới 5-10 giây để khởi động ("loading..."). Kinh khủng hơn, trong suốt một phiên làm việc, người dùng có khi chỉ xài khoảng 100 hàm cơ bản, còn 9.900 hàm nâng cao khác (ví dụ: hàm "In báo cáo", "Xuất file 3D", "Sửa lỗi crash"...)**chả bao giờ được bấm vào**. Công sức đi tìm địa chỉ của 9.900 hàm kia lúc khởi động hoàn toàn là công cốc!

2. Ý tưởng của Lazy Binding: "Nước đến chân mới nhảy"

Để tối ưu tốc độ khởi động app, hệ thống biên dịch GNU quyết định chơi bài Lười: "Lúc bật app lên, tao KHÔNG ĐI TÌM địa chỉ của bất kỳ hàm nào hết. Cứ để app mở lên trong vòng 0.1 giây cái đã. Lúc nào người dùng thực sự bấm nút gọi hàm nào, tao mới vất chân lên cổ đi tìm địa chỉ của đúng hàm đó!"

Cơ chế "vừa lười vừa khôn" này hoạt động cụ thể qua 3 bước:

- Lần đầu tiên gọi hàm (Ví dụ hàm `printf`): 1. Chương trình nhảy vào bảng PLT (Procedure Linkage Table). Do là lần đầu, bảng GOT tương ứng vẫn đang "rỗng" (chưa có địa chỉ thật).
2. Thay vì nhảy đến hàm thật, bảng PLT sẽ dẫn luồng chạy nhảy thẳng vào một hàm cứu hộ của hệ điều hành mang tên Trình phân giải động (Dynamic Resolver).
3. Thằng Resolver này sẽ đi tìm địa chỉ thật của `printf` trên RAM, sau đó nhét cái địa chỉ thật đó vào bảng GOT.
4. Cuối cùng, nó thực thi hàm `printf` cho người dùng.
- Từ lần gọi thứ hai trở đi:
 - Do địa chỉ thật của `printf` đã được lưu sẵn vào bảng GOT từ lần đầu tiên rồi, chương trình chỉ việc nhìn vào GOT và nhảy thẳng tới hàm thật trên RAM mà không cần gọi thẳng Resolver nữa. Tốc độ nhanh như chớp!

💡 Tóm lại:

Lazy binding là kỹ thuật trì hoãn việc tìm kiếm địa chỉ của các hàm cho đến lần đầu tiên hàm đó thực sự được gọi. Nó giúp chương trình khởi động nhanh vèo vèo bằng cách lờ đi việc phân giải địa chỉ của những hàm chưa dùng tới.

Chương này trong sách chuẩn bị nói sâu vào cơ chế hoạt động của cặp bài trùng PLT và GOT để hiện thực hóa cái trò Lazy Binding này. Ông đã sẵn sàng để xem tụi nó tung hứng luồng chạy chưa?

Ý tưởng của LB đó là 1 chương trình có thể gọi từ vài trăm đến vài nghìn hàm từ các thư viện chia sẻ. Bằng cách không cho phân giải hàm cho tới khi nó được gọi, trình biên dịch động có thể tránh việc tái định vị cho rất nhiều hàm không cần thiết tại thời điểm nạp. LB được triển khai với sự kết hợp và tương tác phức tạp của 2 cấu trúc dữ liệu: GOT và Procedure Linkage Table (PLT). Nếu 1 đối tượng gọi bất kì hàm nào được định nghĩa trong thư viện chia sẻ, đối tượng đó sẽ có bảng PLT và GOT riêng biệt. GOT là 1 phần của phân đoạn dữ liệu, còn PLT là 1 phần của phân đoạn code.

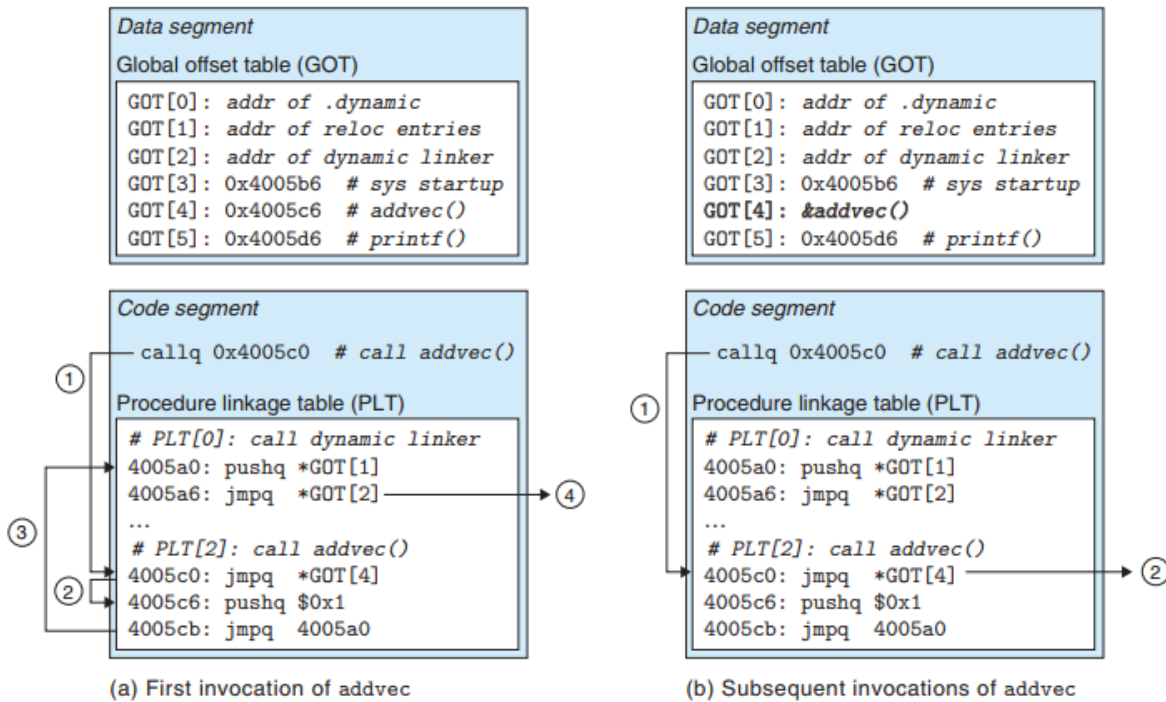


Figure 7.19 Using the PLT and GOT to call external functions. The dynamic linker resolves the address of addvec the first time it is called.

Hình 7.19 cho ta thấy cách mà PLT và GOT phối hợp với nhau để phân giải địa chỉ của hàm tại thời điểm chạy. Đầu tiên, hãy xem nội dung của mỗi bảng.

- **PLT**: là 1 bảng chứa các chỉ mục 16 byte của code. **PLT[0]** là 1 chỉ mục đặc biệt mà nó trỏ đến trình liên kết động. Mỗi hàm của thư viện chia sẻ mà được gọi bởi chương trình thực thi sẽ có 1 chỉ mục PLT riêng. Mỗi chỉ mục này có trách nhiệm gọi hàm được chỉ định trên. **PLT[1]** (không được vẽ trong hình) gọi hàm khởi động hệ thống **__libc_start_main**, có tác dụng khởi tạo môi trường thực thi, và xử lý giá trị trả về. Các chỉ mục bắt đầu từ 2 gọi các hàm được gọi bởi code của người dùng. VD, **PLT[2]** gọi hàm **addvec**.
- **GOT**: Như chúng ta đã biết, GOT là 1 mảng các chỉ mục 8 byte. Khi sử dụng trong việc phối hợp với PLT, **GOT[0]** và **GOT[1]** chứa các thông tin mà trình liên kết động sử dụng khi nó phân giải địa chỉ hàm. **GOT[2]** là chỉ mục trỏ đến trình liên kết động trong module **ld-linux.so**. Các chỉ mục còn lại tương ứng với các hàm được gọi mà địa chỉ cần phải được phân giải tại thời điểm chạy. Mỗi chỉ mục có 1 chỉ mục PLT tương ứng. VD: **GOT[4]** và **PLT[2]** tương ứng với **addvec**. Khi khởi tạo, mỗi chỉ mục GOT đều trỏ vào lệnh thứ 2 của chỉ mục PLT tương ứng. Cách mà PLT và GOT phối hợp trong lần đầu hàm được gọi như sau (7.19 a)):
- Thay vì gọi **addvec**, chương trình gọi **PLT[2]**, là entry của **addvec**.
- Lệnh PLT đầu tiên thực hiện 1 nhảy gián tiếp thông qua **GOT[4]**. Bởi vì mỗi chỉ mục GOT khi bắt đầu đều trỏ đến lệnh thứ 2 trong PLT tương ứng. lệnh nhảy gián tiếp chuyển quyền điều khiển lại cho lệnh kế tiếp trong **PLT[2]**.
- Sau khi đẩy ID của **addvec** (0x1) lên stack, **PLT[2]** nhảy đến **PLT[0]**.

- `PLT[0]` đẩy 1 tham số cho dynamic linker (thông tin về chỉ mục tái định vị) thông qua `GOT[1]` và nhảy đến dynamic linker gián tiếp qua `GOT[2]`. Dynamic linker sử dụng 2 chỉ mục stack để xác định địa chỉ runtime của `addvec`, sau đó ghi đè lên `GOT[4]`, và chuyển quyền điều khiển cho `addvec`.

Trong những lần gọi hàm tiếp theo, `PLT` và `GOT` phối hợp như sau (7.19 b)):

- Chuyển quyền điều khiển cho `PLT[2]` như trước.
- Lần này, lệnh nhảy gián tiếp qua `GOT[4]` sẽ đẩy thẳng đến `addvec` (đơn giản bởi vì sau lần đầu tiên, `GOT[4]` đã bị ghi đè thành địa chỉ của `addvec` thay vì trỏ đến lệnh thứ 2 trong PLT như lúc đầu).

7.13. Chèn thư viện (API hooking).

Linux linker hỗ trợ một kỹ thuật rất mạnh, đó là **chèn thư viện**, cho phép chúng ta chặn bắt các lệnh gọi các thư viện chia sẻ và thực hiện code do chúng ta tự viết. Với hooking, chúng ta có thể trace xem hàm đó được gọi bao nhiêu lần, kiểm tra xem đầu vào và đầu ra của hàm đó là gì, hoặc thay thế hàm đó bằng 1 hàm hoàn toàn khác. Ý tưởng cơ bản đó là: Đối với 1 hàm mục tiêu, ta tạo ra một hàm bao có nguyên mẫu giống với hàm mục tiêu đó. Sử dụng một số cơ chế chèn đặc biệt, bạn có thể lừa hệ thống gọi hàm bao thay vì hàm mục tiêu. Hàm bao sẽ thực hiện logic của nó, sau đó gọi hàm mục tiêu và ép hàm mục tiêu trả về các giá trị cho hàm bao.

(a) Example program `int.c`

```
1  #include <stdio.h>
2  #include <malloc.h>
3
4  int main()
5  {
6      int *p = malloc(32);
7      free(p);
8      return(0);
9  }
```

`code/link/interpose/int.c`

(b) Local `malloc.h` file

```
1  #define malloc(size) mymalloc(size)
2  #define free(ptr) myfree(ptr)
3
4  void *mymalloc(size_t size);
5  void myfree(void *ptr);
```

`code/link/interpose/malloc.h`

(c) Wrapper functions in `mymalloc.c`

```
1  #ifdef COMPILETIME
2  #include <stdio.h>
3  #include <malloc.h>
4
5  /* malloc wrapper function */
6  void *mymalloc(size_t size)
7  {
8      void *ptr = malloc(size);
9      printf("malloc(%d)=%p\n",
10             (int)size, ptr);
11      return ptr;
12  }
13
14  /* free wrapper function */
15  void myfree(void *ptr)
16  {
17      free(ptr);
18      printf("free(%p)\n", ptr);
19  }
20  #endif
```

`code/link/interpose/mymalloc.c`

Figure 7.20 Compile-time interpositioning with the C preprocessor.

Việc chèn có thể xảy ra ở thời điểm biên dịch, thời điểm liên kết, thời điểm chạy. Để khám phá các cơ chế khác nhau này, chúng ta sẽ dùng hình 7.20 a) làm ví dụ. Nó gọi hàm `malloc` và `free` từ thư viện C standard `libc.so`. Lời gọi đến `malloc` cấp phát 1 khối 32 byte trong vùng nhớ heap và trả về con trỏ tới khối đó. Lời gọi tới `free` giải phóng khối được cấp phát. Mục tiêu của chúng ta là dùng chèn để theo dõi lời gọi đến `malloc` và `free` khi chương trình chạy.

7.13.1. Chèn tại thời điểm biên dịch.

Hình 7.20 cho ta thấy cách sử dụng bộ tiền xử lý C (cpp) để thực hiện việc chèn mã tại

thời điểm biên dịch. Mỗi hàm bao trong `mymalloc.c` (hình 7.20 c)) gọi tới hàm mục tiêu, in ra thông tin theo dõi, và trả về. File header cục bộ `malloc.h` (hình 7.20 b)) hướng dẫn cpp thay thế mỗi lời gọi đến hàm mục tiêu thành gọi tới hàm bao. Dưới đây là cách biên dịch và liên kết chương trình.

```
linux> gcc -DCOMPILETIME -c mymalloc.c
linux> gcc -I. -o intc int.c mymalloc.o
```

Việc chèn diễn ra nhờ vào tham số `-I`, định hướng cpp tìm đến `malloc.h` cục bộ (trong thư mục hiện tại), thay vì tìm trong các thư mục hệ thống thông thường. Chú ý rằng các hàm bao trong `mymalloc.c` được biên dịch với chuẩn `malloc.h` header file cục bộ.

Chạy chương trình cho ta thông tin theo dõi sau.

```
linux> ./intc
malloc(32)=0x9ee010
free(0x9ee010)
```

7.13.2. Chèn tại thời điểm liên kết.

Trình liên kết tĩnh của Linux hỗ trợ việc chèn tại thời điểm liên kết với cờ `--wrap f`. Cờ này thông báo cho linker để phân giải tham chiếu tới symbol `f` như là `__wrap_f`, và phân giải tới tham chiếu `__real_f` như là `f`. Hình 7.21 cho ta thấy hàm bao của chương trình ví dụ.

Đây chính là "phép thuật" của thằng Trình liên kết (Linker) mà tôi vừa nhắc đến ở cấp độ 2 (Link-Time Hooking) lúc nãy đấy!

Cái hình này mô tả một cú "Tráo đổi thân phận" cực kỳ hoàn hảo mà thằng Linker làm đạo diễn. Để tôi mổ xẻ cái ma trận lừa đảo này ra cho ông xem, đảm bảo dễ hiểu hơn sách viết ngàn lần.

Trò ảo thuật của cờ `--wrap`

Khi ông biên dịch và liên kết đoạn code này, ông ném cho thằng gcc một cái cờ: `-Wl,--wrap,malloc` (Bảo Linker hãy "gói" cái hàm `malloc` lại).

Ngay khi nhận được lệnh này, thằng Linker sẽ tự động thiết lập một Quy tắc bẻ lái 2 chiều (như đoạn text tiếng Anh ở ảnh đầu tiên mô tả):

1. **Bẻ lái chiều đi (Bắt cóc):** Bất cứ chỗ nào trong code gốc gọi hàm `malloc` (sách gọi là ký hiệu `f`), Linker sẽ lên sửa địa chỉ, bắt nó nhảy thẳng vào hàm `__wrap_malloc` do ông tự viết.
2. **Bẻ lái chiều về (Lối thoát hiểm):** Bất cứ chỗ nào code của ông gọi hàm `__real_malloc` (sách gọi là `__real_f`), Linker sẽ tự động nối dây để nó nhảy về đúng cái hàm `malloc` thật (nằm trong thư viện `libc`).

```

1  #ifdef LINKTIME
2  #include <stdio.h>
3
4  void *__real_malloc(size_t size);
5  void __real_free(void *ptr);
6
7  /* malloc wrapper function */
8  void *__wrap_malloc(size_t size)
9  {
10     void *ptr = __real_malloc(size); /* Call libc malloc */
11     printf("malloc(%d) = %p\n", (int)size, ptr);
12     return ptr;
13 }
14
15 /* free wrapper function */
16 void __wrap_free(void *ptr)
17 {
18     __real_free(ptr); /* Call libc free */
19     printf("free(%p)\n", ptr);
20 }
21 #endif

```

code/link/interpose/mymalloc.c

Figure 7.21 Link-time interpositioning with the `--wrap` flag.

Dưới đây là cách liên kết các file đối tượng vào 1 file thực thi.

```
linux> gcc -Wl,--wrap,malloc -Wl,--wrap,free -o intl int.o mymalloc.o
```

Cờ `-Wl,option` truyền tùy chọn tới linker. Mỗi dấu phẩy `,` ở `option` được thay thế bằng 1 dấu cách . Nên `-Wl,--wrap,malloc` truyền `--wrap malloc` tới linker, và giống như `-Wl,--wrap,free`.

Chạy chương trình được kết quả sau.

```

linux> ./intl
malloc(32) = 0x18cf010
free(0x18cf010)

```

7.13.3. Chèn tại thời điểm chạy.

Việc chèn tại thời điểm biên dịch yêu cầu truy cập tới file nguồn của chương trình, còn chèn tại thời điểm chạy cần truy cập vào file đối tượng tái định vị. Tuy nhiên, có một cơ chế để chèn tại thời điểm chạy chỉ yêu cầu truy cập vào file thực thi. Cơ chế tuyệt vời này dựa trên biến môi trường `LD_PRELOAD` của trình liên kết động.

Nếu `LD_PRELOAD` được đặt thành danh sách các đường dẫn đến các thư viện chia sẻ (phân tách bằng dấu cách), sau đó nạp và thực thi chương trình, linker `ld-linux.so` sẽ tìm kiếm các thư viện trong `LD_PRELOAD` trước, khi nó phân giải các tham chiếu chưa được định nghĩa. Với cơ chế này, bạn có thể chèn vào bất kì hàm trong bất kì thư viện

nào, bao gồm cả `libc.so`, khi bạn nạp và thực thi bất kì file thực thi nào.

code/link/interpose/mymalloc.c

```
1  #ifdef RUNTIME
2  #define _GNU_SOURCE
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <dlfcn.h>
6
7  /* malloc wrapper function */
8  void *malloc(size_t size)
9  {
10     void *(*malloclp)(size_t size);
11     char *error;
12
13     malloclp = dlsym(RTLD_NEXT, "malloc"); /* Get address of libc malloc */
14     if ((error = dlerror()) != NULL) {
15         fputs(error, stderr);
16         exit(1);
17     }
18     char *ptr = malloclp(size); /* Call libc malloc */
19     printf("malloc(%d) = %p\n", (int)size, ptr);
20     return ptr;
21 }
22
23 /* free wrapper function */
24 void free(void *ptr)
25 {
26     void (*freelp)(void *) = NULL;
27     char *error;
28
29     if (!ptr)
30         return;
31
32     freelp = dlsym(RTLD_NEXT, "free"); /* Get address of libc free */
33     if ((error = dlerror()) != NULL) {
34         fputs(error, stderr);
35         exit(1);
36     }
37     freelp(ptr); /* Call libc free */
38     printf("free(%p)\n", ptr);
39 }
40 #endif
```

code/link/interpose/mymalloc.c

Figure 7.22 Run-time interpositioning with LD_PRELOAD.

Hình 7.22 cho ta thấy hàm bao của `malloc` và `free`. Trong mỗi hàm bao, lời gọi đến `dlsym` trả về con trỏ tới hàm mục tiêu trong `libc`. Hàm bao sau đó sẽ gọi đến hàm mục tiêu, và trả về.

Dưới đây là cách để xây bulid thư viện chia sẻ chứa các hàm bao.

```
linux> gcc -DRUNTIME -shared -fpic -o mymalloc.so mymalloc.c -ldl
```

Còn đây là cách để biên dịch chương trình chính.

```
linux> gcc -o intr int.c
```

Đây là cách để chạy chương trình từ bash shell.

```
linux> LD_PRELOAD="./mymalloc.so" ./intr
malloc(32) = 0x1bf7010
free(0x1bf7010)
```

Chú ý rằng bạn có thể sử dụng `LD_PRELOAD` để chèn vào các lệnh gọi thư viện trong bất kì file thực thi nào.

```
linux> LD_PRELOAD="./mymalloc.so" /usr/bin/uptime
malloc(568) = 0x21bb010
free(0x21bb010)
malloc(15) = 0x21bb010
malloc(568) = 0x21bb030
malloc(2255) = 0x21bb270
free(0x21bb030)
malloc(20) = 0x21bb030
malloc(20) = 0x21bb050
malloc(20) = 0x21bb070
malloc(20) = 0x21bb090
malloc(20) = 0x21bb0b0
malloc(384) = 0x21bb0d0
20:47:36 up 85 days,  6:04,  1 user,  load average: 0.10, 0.04, 0.05
```

7.14. Các công cụ thao túng file đối tượng.

Có rất nhiều các tool trên Linux giúp bạn hiểu và thao túng các file đối tượng. Thực tế, bộ đóng gói GNU `binutils` là có ích và chạy trên tất cả các nền tảng Linux.

- **AR**: Tạo các thư viện tĩnh, chèn, xóa, liệt kê các phần tử bên trong nó.
- **STRINGS**: Liệt kê tất cả các chuỗi in được có trong 1 file đối tượng.
- **STRIPS**: Xóa thông tin bảng symbol trong 1 file đối tượng.
- **NM**: Liệt kê các symbol được định nghĩa trong symbol table của 1 file đối tượng.
- **SIZE**: Liệt kê tên và kích thước của các section có trong 1 file đối tượng.
- **READELF**: Hiển thị cấu trúc hoàn chỉnh của 1 file đối tượng, bao gồm tất cả các thông tin có trong ELF header.
- **OBJDUMP**: Mẹ đẻ của tất cả các tool nhị phân, có thể hiển thị tất cả thông tin có trong 1 file đối tượng. Hàm hữu dụng nhất của thư viện này là giải mã các lệnh nhị phân trong section `.text` (tức là hiển code asm).